

# Empirical evaluation of the performance of data sampling and feature selection techniques for software fault prediction

Sonika Chandrakant Rathi <sup>a</sup>, Sanjay Misra <sup>b,c,\*</sup>, Ricardo Colomo-Palacios <sup>d</sup>, R. Adarsh <sup>a</sup>,  
Lalita Bhanu Murthy Neti <sup>a</sup>, Lov Kumar <sup>e</sup>

<sup>a</sup> BITS Pilani Hyderabad, India

<sup>b</sup> Østfold University College, Halden, Norway

<sup>c</sup> Institute for Energy Technology, Halden, Norway

<sup>d</sup> Universidad Politécnica de Madrid (UPM), Spain

<sup>e</sup> National Institute of Technology, Kurukshetra, India

## ARTICLE INFO

### Keywords:

Data sampling  
Feature selection techniques  
Fault prediction models  
Machine learning

## ABSTRACT

**Context:** The application of Software Fault Prediction (SFP) in the software development life cycle to predict the faulty class at the early stage has piqued the interest of various scholars. In the SFP domain, during research analysis, it got realized that there has been very little work instigated on addressing both class imbalance and feature redundancy problems jointly to enhance the performance and prediction accuracy of SFP models. It has been perceived in the literature survey the study of droughts with the comprehensive comparative analysis of different sampling and feature selection strategies together.

**Objective:** This research builds an extensive assessment of distinct combinations of different feature selection and sampling approaches, to effectively overcome the problems of class overlap, class imbalance, and feature redundancy. The objective is to determine the best combination that will produce results with a higher degree of accuracy and an effective SFP model.

**Method:** Considering the above erudition, the study has applied 8 different sampling techniques along with 10 feature selection algorithms against 56 open-source projects. The comparative analysis is performed against 5346 variants of input datasets by applying 8 different classifiers to predict the faulty class. In addition, the research paper presents an intensive assessment and performance of these techniques individually against all the input projects. We have considered accuracy and Area Under the ROC (receiver operating characteristic curve) Curve (AUC) performance metrics to compare the performance of different models developed using the classification algorithm.

**Result:** For each project in the proposed work, we evaluated a total of 792 combinations that were produced using 10 feature selection methods, 1 all metrics dataset, 8 sampling methods, 1 original, unsampled dataset, and 8 classifiers. The empirical result indicates that, against 21 projects out of 54 projects, Synthetic Minority Over Sampling Technique Edited (SMOTEE) with correlation-based feature selection (FS2) combination outperformed with the highest AUC value which is 38.89 % of projects. Additionally, according to experimental results, the highest AUC values were attained by 24.07 % of projects using the SMOTEE, FS2, and RF combination.

**Conclusion:** The results of the statical analysis test reveal that 93.42 % of the combinational pairs of different sampling and feature selection approaches demonstrated a significant variance in the performance of the distinct combinations of sampling and feature selection techniques. The empirical result indicates the performance of the SFP Model is adversely impacted by class imbalance and irrelevance. The outcome indicates for more than 75% of projects, the performance of trained models improved with an AUC value between a range of 0.805 to 0.99 post-application of sampling and feature selection strategies, in comparison without the use of feature selection and sampling techniques.

\* Corresponding author at: Østfold University College, Halden, Norway.

E-mail addresses: [sonika.rathi@pilani.bits-pilani.ac.in](mailto:sonika.rathi@pilani.bits-pilani.ac.in) (S.C. Rathi), [sanjay.misra@hiof.no](mailto:sanjay.misra@hiof.no) (S. Misra), [ricardo.colomo@upm.es](mailto:ricardo.colomo@upm.es) (R. Colomo-Palacios), [f20190230@hyderabad.bits-pilani.ac.in](mailto:f20190230@hyderabad.bits-pilani.ac.in) (R. Adarsh), [bhanu@hyderabad.bits-pilani.ac.in](mailto:bhanu@hyderabad.bits-pilani.ac.in) (L.B.M. Neti), [lovkumar@nitkkr.ac.in](mailto:lovkumar@nitkkr.ac.in) (L. Kumar).

<https://doi.org/10.1016/j.eswa.2023.119806>

Received 14 October 2022; Received in revised form 6 February 2023; Accepted 4 March 2023

Available online 17 March 2023

0957-4174/© 2023 Published by Elsevier Ltd.

## 1. Introduction

Software quality is an important facet of maintaining software consistency for organizations. Preserving software consistency in terms of software quality becomes essential as well as challenging; during the scale of a software system, the size and complexity of software increases, which engenders more faults. Furthermore, the faults existing in the software system affect the quality and dependability of the software. Faults are considered an overhead for the system and most organizations and companies want to predict the defects in the code or identify fault-prone modules in the early stage of software development systems (Menzies et al., 2010). Even minor faults can lead to erroneous outputs and unexpected behavior of the program, thus leading to the failure of the project. To be in line with the software engineer's principal goal, to deliver and ensure error-free software to end-users, the software must be tested properly (Menzies et al., 2010). Several ways of detecting faults in the early stages of software development have been presented by researchers in the past (Singh & Malhotra, 2012). Learning signposts detection of faults in early-stage helps to control quality, manage delivery timelines and minimize post-production bugs, thereby allowing for on-time distribution to customers. The common use case among all researchers and the software community is software fault prediction to identify whether a module is defective or not by using Data Mining (DM) and Machine Learning (ML) techniques (Bhandari, Kumar, & Sangal, 2021).

The DM and ML techniques can help in predicting whether a class is bug-prone or not, which will help the software engineers to give more attention to bug-prone modules and thus saving a lot of time and cost. Optimization in time & cost will lead to minimizing the software maintenance outlay and an increase in customer satisfaction. In this paper, we employed the eight most prominent machine learning classifiers as suggested by several researchers (Lessmann, Baesens, Mues, & Pietsch, 2008; Pandey, Mishra, & Tripathi, 2021; Son et al., 2019) in the domain of Software Fault Prediction using ML. The Gaussian Naïve Bayes (GNB), Logistic Regression (LOGR), Decision Tree (DT), Random Forest (RF), Support Vector Machine (SVM) with radial basis function kernel function (SVCR), SVM with linear kernel function (SVCL), Bagging (BAG), and AdaBoost (ADAB) has been employed to train Software Fault Prediction (SFP) model and predict the faulty class in the datasets. All of the chosen classification techniques are tested against 54 open-source object-oriented projects.

Object-oriented software methodologies are widely used in software development. Software metrics are used for developing fault prediction models, their performance will also depend on these source code metrics (Basili, Briand, & Melo, 1996; Chidamber & Kemerer, 1994). Hence, selecting proper subsets of software metrics will affect the overall performance of the prediction model (Rathore & Gupta, 2014). One of the purposes of this study is to identify the impact of feature selection methods on fault predictability. The performance of the SFP models depends on these metrics; it is essential to identify the right set of metrics to optimize the performance of implemented models to detect faulty classes. Thus, in this paper, we have applied five feature ranking and five feature subset selection techniques over a variety of datasets to identify the best software metrics/features that are useful to train the SFP models.

The input dataset contains a varied distribution of instances, majority and minority samples, etc., hence, it impacts the performance of SFP models. As a result, the training dataset's nature influences the classifier's performance (Elahi, Kanwal, & Asif, 2020). Generally, in software project metrics, the number of non-faulty cases outnumbers the number of defective cases, resulting in a class-imbalanced challenge in SFP models. SFP models trained on these class-imbalanced datasets may favor non-defective examples while misclassifying defective ones (Pak, Wang, & Su, 2018). After these training datasets have been balanced, the developed SFP models may achieve good prediction performance (Gong, Jiang, & Jiang, 2019; Tantithamthavorn, Hassan,

& Matsumoto, 2018). Consequently, the problem of class imbalance is one of the major variables impacting the performance of SFP models, and a growing number of researchers are striving to overcome it. Another objective of this paper is to tackle the class imbalance problem, which can impede the SFP model's performance. Thus, before applying the feature selection techniques we balance the datasets using eight different sampling methods to solve the class imbalance problem.

Our research analysis in the field of SFP divulges there have been very few studies (Joon, Tyagi, & Kumar, 2020; Pandey & Tripathi, 2021; Yohannese & Li, 2017) conducted against addressing both class imbalance and feature redundancy problems collectively to improve the performance and prediction accuracy of the SFP model. Yohannese and Li (2017) engaged SMOTE Sampling technique and Correlation-based feature selection (CFS), Principal Component Analysis (PCA), & Information Gain (IG) feature selection techniques in their study. Joon et al. (2020) employed the SMOTETomek sampling technique and simple majority voting feature selection. In the study performed by Pandey and Tripathi (2021), evaluated sampling techniques are random undersampling, class balancer, SMOTE, & Spread Subsample Technique, they also employed the information gain feature selection technique. However, the above-mentioned study work does not include thorough experimentation of a combination of various sampling procedures and feature selection techniques on the performance of SFP models. Additionally, limited datasets (mostly on an average of 4 to 6) were used for the experiments in the previous studies that combined sampling and feature selection techniques.

It is necessary to determine whether the combined approach significantly improves the performance and accuracy of the developed SFP model because the implementation of the sampling and feature selection approach will also increase complexity, this paper analyses the performance of the developed SFP model with/without feature selection and sampling approaches. This paper conducts extensive experimentation to identify the best-performing combination to optimize the implementation cost and efficacy of the developed SFP model. Thus, we have applied 10 feature selections + 1 all feature metric, 8 sampling techniques + 1 original dataset, and 8 classifiers, against 54 different projects holding a different percentage of faulty classes. The main goal of this study is to jointly address the class imbalance and feature redundancy issues, as well as to draw a general conclusion regarding the best-performing combination, and optimize, and enhance the outcomes of the proposed SFP model over others.

Our major contributions to the paper are as follows:

- We are the first to thoroughly examine the combined approach with 99 different combinations of feature selection and sampling techniques jointly ((10 Feature selection + 1 All metrics) \* (8 sampling approaches + 1 original/unsampled dataset)) against 54 open-source datasets, with the employment of 8 well-known classifiers, to draw a generalized conclusion on the most efficient combination
- We carried out the statical analysis test to verify the performance difference of various combinations of feature selection and sampling techniques
- As the implementation of the sampling and feature selection approach will also increase complexity, we compare the performance of the developed SFP model with/without feature selection and sampling approaches to determine whether the combined approach significantly improves the performance and accuracy of the developed SFP models

The respective Research Questions (RQs) and their sub questions are justified in this study:

*The primary RQ: What effect do the use of different combinations of feature selection and sampling approaches have on the performance of the developed SFP model?* A total of 54 open-source datasets were used in this study, and 99 unique combinations were examined using 10 feature

selection methods, 1 all metrics dataset, 8 sampling methods, and 1 original i.e. unsampled dataset, by employing 8 classifiers. The study evaluated the efficacy of the built models and determined the overall top-performing combinations.

The corresponding sub-questions are:

- *RQ1: Is the outcome of feature-ranking and feature-subset selection strategies statistically equivalent?*  
We carried out extensive experimentation of 5 feature ranking and 5 feature subset selection methods and 1 All feature metrics against 54 open-source object-oriented datasets in this study to draw general conclusions and determine the best-performing feature selection approach individually.
- *RQ2: How powerful are the various feature selection methods in terms of optimizing the performance of the SFP model ?*  
In this study, we have investigated the performance of 11 feature selections (1 for all feature metrics) with a combination of 8 sampling algorithms and 1 unsampled i.e. original dataset against 54 open-source datasets.
- *RQ3: Which sampling method gives efficient and effective performance?*  
In order to reach a general conclusion about the sampling method that performs the best overall, we employed 8 sampling methods and tested their effectiveness with the original dataset against 54 input datasets.
- *RQ4: What is the impact of sampling techniques on the predictability of SFP models?*  
The study examines how well a set of eight classifiers and nine sampling techniques performs together (1 unsampled or original dataset).
- *RQ5: What effect do different classifiers have on predicting faulty source code?*  
The paper investigates and validates whether the performance of eight classifiers produces the same outcome with all employed combinations of feature selection and sampling techniques.
- *RQ6: Is the outcome of various feature selection methods combined with different sampling techniques statistically equivalent?*  
In the proposed research work, we have examined the utilized feature selection methods after combining 8 sampling approaches and 1 unsampled dataset performance.

In addition, to ensure fair predictions, the study conducts statistical analysis of the engendered software fault prediction models. The performance and reliability of these models were evaluated using the performance parameters: accuracy and AUC.

The paper is organized as follows: Section 2 provides a summary of related work in the field of software fault prediction. Section 3 explains the used methodologies in our experimentation. The research framework, the results & analysis of combined approaches, and the model performance is presented in Section 4. Section 5 covers the comparative analysis of combined techniques (feature selection and sampling). And the discussion of results for outcomes of combined techniques & individual performance, threats to validity and conclusion & future work are depicted in Sections 6–8 respectively.

## 2. Related work

This section represents the related work in four categories. First, it enlightens the research done in the field of software fault prediction to forecast defective software. In particular, we have highlighted the study done in the SFP domain using data mining and machine learning techniques. The study that was done on software metrics is covered in more detail in the second section. This study aimed to identify the best set of metrics for software fault prediction. Later, the third part of this section focuses on the studies conducted to take care of feature redundancy. The fourth part covers the study conducted in the field

of addressing class imbalance problems to construct fault-proneness prediction models using various sampling approaches.

A review of different databases (IEEE, Science Direct, Elsevier, Springer, ACM, Scopus, etc.) reveals that numerous studies have been done on examining feature selection and sampling methods independently. However, only a few studies cover feature selection and sampling together, and to the best of our knowledge, those studies lack thorough experimentation with various combinations and that relevant research has been done against limited datasets (it is expressly mentioned in the introduction section's fifth paragraph of our paper).

We have included the references that discuss feature selection and sampling together, as well as some recent references from the best publications that discuss feature selection and sampling separately (we did not feel it was necessary to include all the studies that were restricted to individual techniques). Lastly, in the fifth subsection, we have derived the motivation for our research.

### 2.1. Research conducted in SFP model development

Literature has produced several works in the field of software fault prediction (Ji, Huang, Wu, Hui, & Zheng, 2019; Malhotra & Jain, 2012; Zhou, Sun, Xia, Li, & Chen, 2019). The main purpose of SFP models is to predict a class as faulty or non-faulty. Predicting faults in a software module helps the software test resources to be utilized more effectively and efficiently in quality assurance. Malhotra and Jain (2012) proposed six machine learning methods and statistical methods to predict the faulty class. Ji et al. (2019) suggested a weighted Naïve Bayes on 10 different datasets, and the result shows the proposed model performs significantly better. The sequential ensemble model on PROMISE and ECLIPSE repository datasets is studied by Mangla, Sharma, and Mohanty (2021). Nevendra and Singh (2021) suggested a new approach to AdaBoost and showed the proposed method improves the prediction efficiency (McCabe, 1976). Other proposed software defect prediction models are SVM by Hou and Li (2009) and Kulamala, Kumar, and Mohapatra (2021), and the Random forest classifier-based approach by Dam et al. (2018); & Zhou et al. (2019), Multilayer Perceptron, J48, two-hybrid search-based algorithms by Rhmann, Pandey, Ansari, and Pandey (2020), Neural network approach by Zong, Wang, and Xie (2018), etc. In our present work, we have employed the most studied eight classifiers: GNB, LOGR, DT, RF, SVC, SVCL, BAG, and ADAB, to train the SFP model and investigate the performance difference post-application of feature selection and sampling approaches.

### 2.2. Software metrics

A considerable amount of work has been performed in the field of software metrics and defect prediction. However, it is been difficult to select the right set of metrics that would be effective for software defect prediction. Chidamber and Kemerer (1994), Henderson-Sellers (1996), and McCabe (1976) proposed various software metrics in their study. Jureczko (2011) tries to study the importance of various metrics in defect prediction and performs correlation analysis to understand the relationship between several defects and software metrics. The paper classified the study into two categories, namely static code components and change metrics. The analyzed metrics were CK Metrics which include Weighted Methods per Class (WMC), Depth of Inheritance Tree (DIT), Number of Children (NOC), Coupling between object classes (CBO), Lack of Cohesion of Methods (LCOM), etc. Other metrics studied were Martin's metrics which cover Afferent Coupling (Ca) & Efferent Coupling (Ce). Halstead metrics consider the program length, program volume, program level and difficulty, intelligent content, and programming effort. Misra, Adewumi, Fernandez-Sanz, and Damasevicius (2018) have presented a set of cognitive complexity metrics that can be used to assess OO software projects. These metrics consider the complexity of the method, the message, the attribute, the weighted class, and the code. Crasso, Mateos, Zunino, Misra, and Polvorin (2016)

conducted research on the software metric used to evaluate cognitive complexity for the Java language. Based on a description of the fundamental control structures found in Java systems, the proposed metric was developed.

### 2.3. Feature redundancy problem

The speed and accuracy of the trained classifiers can be affected by irrelevant/redundant features. In addition to this, the literature survey shows that feature selection leads to better performance of the classification model (Rathore & Gupta, 2014). Feature selection techniques are applied during the pre-processing data stage to increase the performance of SFP models. Over the years, constant efforts have been made to improve the statistical accuracy of models. Larger datasets would necessitate extra time and increased compute cycles to train the SFP model, which could impact prediction accuracy too. For this reason, attribute selection is used. Here, in this paper, we employed 5 feature ranking and 5 feature subset selection techniques and carried out a comparative performance analysis.

Generally, any model performance depends largely on choosing appropriate classifiers, relevant metrics, and the quality and consistency of training data. The filter technique and the wrapper approach are the two types of attribute selection algorithms (Tan, Steinbach, & Kumar, 2005). Filter approaches involve selecting the attribute set before running the algorithm. The wrapper method uses a target algorithm to find an optimal feature subset. The correlation-based and consistency-based feature ranking methods, as well as five feature selection techniques, were investigated by Kondo, Bezemer, Kamei, Hassan, and Mizuno (2019) against three datasets using five supervised models and five unsupervised models.

Gao et al. have applied some feature selection techniques for fault prediction on large legacy software systems in telecommunications (Gao, Khoshgoftar, Wang, & Seliya, 2011). Wang et al. evaluated various ensembles of feature selection algorithms (Wang, Khoshgoftar, & Napolitano, 2010). It was shown that sometimes ensembling some algorithms can prove to be better than ensembling all algorithms. Rodriguez et al. evaluated different filter approaches and wrapper approaches (Rodríguez, Ruiz, Cuadrado-Gallego, & Aguilar-Ruiz, 2007) and showed that wrapper methods involve higher computation costs, but they outperformed filter methods. A comprehensive inspection of feature selection algorithms is published by Liu and Yu (2005). Laradji, Alshayeb, and Ghouti (2015) recommended Software defect prediction using ensemble techniques on selected features, and the result shows the remarkable performance of the proposed model. Hall and Holmes (2003) reported a comparative inspection of six different attribute selection strategies that create a feature-ranked list. The findings of the experiments suggest that attribute selection improves the performance of learning algorithms in general. Wang et al. (2010) examined multiple feature selection techniques using SVM and Naïve Bayes classifiers. Tubishat, Idris, Shuib, Abushariah, and Mirjalili (2020), proposed an improved salp swarm algorithm to select the optimal subset of features. The performance of feature selection was improved in the study using opposition-based learning and a new local search algorithm.

### 2.4. Class imbalance problem

Software defect datasets are made up of high dimensionality of data and a poor ratio of defect-prone to non-defect-prone classes leading to class imbalance (Chen et al., 2014; Gong et al., 2019; Tantithamthavorn et al., 2018). These datasets are heavily skewed toward non-defect-prone modules, with a very less number of defect-prone modules. This impacts the performance of SFP models. As a result, data balancing using sampling methods is applied to correct the skewed nature of defect datasets, allowing SFP models to be built on balanced data to improve prediction capability (Cynthia & Ripon, 2019; Khuat & Le, 2019). In this paper, we addressed the class imbalance problem by

applying 8 sampling techniques before feature selection employment. To construct the SFP model, Yohannese and Li (2017) used the feature selection approach and SMOTE. SMOTE is a form of oversampling technique that is used to balance data. In addition, they utilized an ensemble approach to predict the faulty class by integrating the outputs of all trained classifiers. Weiss, McCarthy, and Zabar (2007) examined methods for dealing with skewed class distributions and non-uniform classification costs. As a part of this study, methods to reduce costs and sampling (oversampling and undersampling) were investigated. The paper concluded that cost-sensitive learning seems to outperform sampling techniques (Weiss et al., 2007). The performance of the sampling technique used depends on the nature of the dataset too.

The authors in Elahi et al. (2020) explored and studied the comparison of several ensemble strategies to increase the performance of a prediction model. Also, they employed different sampling approaches, such as SMOTE, random undersampling, and random oversampling, to address the problem of class imbalance. Furthermore, they implemented the model averaging method using voting and staking, and the result indicated that the method of model averaging outperformed the other. Gong et al. (2019) to solve the class imbalance problem proposed the K-Means Cluster-based Over-sampling with a noise filtering approach and revealed their proposed model to obtain better Recall and balance values. Malhotra and Kamal (2019) performed the study to improve software defect prediction by handling the imbalanced data using different oversampling techniques and also proposed the SPIDER3 sampling technique. They compared the performance and impact of different oversampling techniques over SFP models. Joon et al. (2020) presented the optimized software fault prediction by combining basic noise removal, imbalanced class distribution, and software metrics selection strategies. They employed the SMOTETomek resampling technique in their proposed study. A recent extensive study was performed by Pandey and Tripathi (2021) towards dealing with noise and class imbalance problems in software defect prediction. They did compare the performance of the random undersampling technique, SMOTE, Class Balancer, and Spread Sub-Sampling, and observed random undersampling technique achieved the best result.

Table 1 highlights studies conducted by several researchers who employed various methodologies to develop SFP models, as well as the numerous evaluation factors taken into account when selecting successful fault prediction models.

### 2.5. Motivation for the study

Based on the discussion in previous sections, it is observed that there has been profound research conducted in the area of SFP to address the class imbalance using sampling techniques and feature redundancy problems using feature selection separately. However, further analysis indicates there is very little investment seen in handling both class imbalance and feature redundancy problems together to achieve greater performance and prediction accuracy of the SFP model. Although some studies were conducted that included both, against very limited datasets; an in-depth comparison of different sampling and feature selection strategies is lacking. As a result, there is a need for in-depth research to evaluate the performance of SFP models by combining sampling and feature selection techniques. This point is our major motivation for our present work and leads us to endow our focus on implementing the proposed model to address the substantial gap identified to extemporize the performance and predictability of the SFP model by engaging sampling and feature selection techniques jointly. This research work exploits the implication of 8 sampling strategies and 10 feature selection techniques to tackle the class imbalance and feature redundancy problem. The performance of developed SFP models is analyzed using AUC and accuracy metrics developed with/without the implication of sampling and feature selection techniques.

**Table 1**  
Studies carried out in the SFP domain.

| Author                                                    | Dataset                                                                                   | Method Used                                                                                                                                                                                                      | Metric Used                                                                                                                         | Limitation                                                                                                                                             |
|-----------------------------------------------------------|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Khuat and Le (2019)                                       | PROMISE repository                                                                        | Support vector machines (SVM), multilayer perceptron (MLP), Bayesian networks, K-nearest neighbor (KNN), and decision tree J48                                                                                   | Lines Of Code-based measures, McCabe and Halstead, F-score                                                                          | Comparative analysis with other sampling techniques, feature redundancy problem                                                                        |
| Yohannese and Li (2017)                                   | PROMISE repository                                                                        | Sampling approaches SMOTE, Feature selection techniques: CFS, PCA, and IG, Classifiers: Naive Bayes, NN, SVM, RF, KNN, DT, DTr and RTr                                                                           | Static Code Metrics (SCM), CK object-oriented software Metrics                                                                      | Comparative study of the impact of various feature selection techniques on the performance of the SFP model, analysis of different sampling techniques |
| Karim, Warnars, Gaol, Abdurachman, Soewito, et al. (2017) | NASA dataset from PROMISE repository                                                      | Machine learning technique                                                                                                                                                                                       | Lines of Code (LOC), Cyclomatic Complexity Number (CCN)                                                                             | Class imbalance, redundancy issues                                                                                                                     |
| Arshad, Riaz, Jiao, and Murthy (2018)                     | NASA & Eclipse dataset                                                                    | Semi-supervised deep Fuzzy C-Mean (DFCM) clustering-based feature extraction technique                                                                                                                           | Lines of Code (LOC), Cyclomatic Complexity Number (CCN)                                                                             | Investigation over relationships between the different feature selection approaches                                                                    |
| Mohapatra and Ray (2018)                                  | Class imbalance data                                                                      | Group Search Optimization Genetic Algorithm, SVM classification                                                                                                                                                  | Average Percentage of Faults Detected (APFD), Problem Tracking Re-ports (PTR)                                                       | Comprehensive analysis with sampling techniques, an extension of the findings to other software systems                                                |
| Ranjan, Kumar, and Kumar (2017)                           | NASA data sets, PROMISE repository, Eclipse data set                                      | Soft computing, data mining, and machine learning-based approaches                                                                                                                                               | Fuzzy logic and McCabe based metrics                                                                                                | Data imbalance, redundancy study                                                                                                                       |
| Turabieh, Mafarja, and Li (2019)                          | 19 real-world software projects from the PROMISE repository                               | Binary Genetic Algorithm (BGA), Binary Particle Swarm Optimization (BPSO), Binary Ant Colony Optimization (BACO), layered recurrent neural network (L-RNN)                                                       | Ck Metrics                                                                                                                          | Investigation of the performance of different classifiers, class imbalance                                                                             |
| Tantithamthavorn, Hassan, and Matsumoto (2020)            | 101 Open source datasets                                                                  | Sampling approaches: Over-sampling, Under sampling, SMOTE and Bootstrap Random Over-Sampling                                                                                                                     | Ck Metrics                                                                                                                          | Comparative analysis of the impact of various feature selection techniques on the performance of the SFP model                                         |
| Zong et al. (2018)                                        | Git repository has been used to extract the v4–v5, v2–v5 of android for defect prediction | Random Forest, Multilayer Perceptron, J48, two-hybrid search based algorithms: GFS Adaboost-c and GFS LoogitBoost-c                                                                                              | LOC, Code Churn                                                                                                                     | Study over data imbalance                                                                                                                              |
| Shao, Liu, Wang, and Li (2020)                            | PROMISE and NASA dataset                                                                  | Correlation weighted class association rule mining (CWCAR)                                                                                                                                                       | LOC, CK, Martins, QMOOD, McCabe's CC, Bug                                                                                           | Comparative analysis of the impact of various feature selection techniques on the performance of the SFP model                                         |
| Aziz, Khan, and Nadeem (2019)                             | PROMISE Dataset                                                                           | ANN                                                                                                                                                                                                              | Ck-Metrics                                                                                                                          | Regression-based machine learning techniques, feature redundancy                                                                                       |
| Shatnawi (2017)                                           | Open-source system — Apache Ant, Apache Camel, jEdit, Apache Lucene, Apache Synapse       | Naïve Bayes (NB), logistic regression (LR), the nearest neighbors (KNN), and C4.5 decision trees                                                                                                                 | CK metric, ROC analysis                                                                                                             | Investigation of a larger number of metrics using a variety of sample and feature selection strategies                                                 |
| Kondo et al. (2019)                                       | 3 Datasets (PROMISE, NASA, AEEEM)                                                         | Feature selections techniques: correlation-based and consistency-based and 5 feature selection techniques Classifiers: five supervised models and five unsupervised models                                       | CK, OO, McCabe, Halstead, number of previous defects change features, complexity code change features, churn & entropy of CK and OO | Limited datasets and utilization and assessment of combined approach with various sampling approaches                                                  |
| Elahi et al. (2020)                                       | PROMISE repository                                                                        | Sampling approaches: SMOTE, random under-sampling, and random oversampling Classifiers: Decision trees, logistic regression, Naive Bayes, multinomial naive Bayes, and K nearest neighbor Model averaging method | CK-Metrics                                                                                                                          | Investigation of other sampling techniques, feature selection techniques                                                                               |
| Gong et al. (2019)                                        | NASA, AEEEM, ReLink, and SOFT LAB                                                         | Sampling technique: SMOTE, BSMOTE, K-Means Cluster-based Over-sampling with noise filtering approach Classifiers: Naïve Bayes, Random Forest, SVM, Logistic regression, and Decision tree                        | LOC, cyclomatic complexity                                                                                                          | Feature selection approaches                                                                                                                           |

(continued on next page)

**Table 1** (continued).

| Author                                        | Dataset                                       | Method Used                                                                                                                                                                                                                     | Metric Used                 | Limitation                                                                                                                       |
|-----------------------------------------------|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Malhotra and Kamal (2019)                     | NASA datasets                                 | Sampling techniques: SMOTE, Adaptive Synthetic sampling technique, Safe-Level-SMOTE, Selective Preprocessing of Imbalanced Data (SPIDER), and SPIDER2. SPIDER3. Classifiers: J48, Random Forest, Naïve Bayes, Adaboost, Bagging | ALOC, cyclomatic complexity | Analysis of undersampling methods                                                                                                |
| Joon et al. (2020)                            | NASA, Promise, and Github repository datasets | SMOTETomek Simple majority voting feature selection Classifiers: logistic regression, random forest (RF), SVM, naïve Bayes                                                                                                      | LOC, cyclomatic complexity  | Performance comparison of other sampling techniques & feature selection techniques                                               |
| Liu et al. (2015)                             | 8 datasets (Eclipse and NASA)                 | Feature selection: wrapper-based feature ranking techniques Sampling method: random under sampling Classifiers: Naïve Bayes, Decision Tree and K-Nearest Neighbors                                                              | CK Metrics, LOC             | Experiments were carried out against limited datasets, and considered only two feature ranking methods and one sampling approach |
| Balogun et al. (2020)                         | NASA datasets                                 | Feature selection method: CFS Sampling approach: SMOTE and Classifiers: Bayesian Networks and Decision Tree                                                                                                                     | LOC Metrics                 | Limited number of combinations executed against limited datasets                                                                 |
| Thaher, Mafarja, Abdalhaq, and Chantar (2019) | 14 datasets (PROMISE)                         | Classifier: KNN, Sampling: SMOTE, Feature Selection: Wrapper-based Feature Selection                                                                                                                                            | CK Metrics                  | Utilization and assessment of various sampling approaches                                                                        |
| Putri et al. (2017)                           | 4 datasets (NASA MDP)                         | Feature selection: chi-square, information gain, and Relief algorithm Sampling methods: SMOTE, and Random Under-Sampling Classifier: Naïve Bayes classifier                                                                     | Loc Metrics                 | Limited classifiers, feature selection and sampling methods combinations                                                         |

### 3. Used methodologies in our experimentation

This section enlightens on the components required for our study. We are providing the information on datasets, feature selection techniques, sampling strategies and classification approaches. This section also discusses statistical tests and performance parameters.

#### 3.1. Experimental data-sets

In this study, we have used 54 open-source object-oriented software project datasets<sup>1</sup> shown in Table 2.

The considered input or independent variables of the proposed fault prediction model are twenty well-known object-oriented software metrics mentioned in Table 3 (Kumar, Sripada, Sureka, & Rath, 2018); these metrics are comprised of size, inheritance, coupling, cohesion, class, and children. The dependent variable of our proposed fault prediction model is a class with two values, either faulty or non-faulty. Table 4 displays the descriptive statistics for each project's metrics that were determined to be the most relevant and efficient post-application of feature selection techniques (discussed in Section 4.2), with a 95% confidence level. Table 4, depicts the descriptive statistics of the Project 1 dataset, for the software metrics, WMC ranges from 8.8 to 11.4, and for CBO it ranges between 8.5 to 12.7. Similarly, ranges for other metrics of the relevant projects can be seen.

#### 3.2. Feature selection techniques

The following attribute selection techniques have been implemented and compared as a part of this study. Table 5, shows the naming convention of the different feature selection methods employed in this paper.

The feature ranking methods considered in this study are oneR, Relief, Gain Ratio, Info gain, and Chi-squared test. oneR is a rule-based algorithm that generates a rule for predictor variables and ranks the features based on classification rates. The relief algorithm takes a filter-based approach to feature selection; the variation of information gain

technique that helps in the reduction of bias is called Gain Ratio. This technique accounts for the intrinsic information of a split. Information gain indicates the reduction in entropy, and it is calculated by comparing the entropy of the dataset before and after a transformation. The chi-square statistic value can be used to rank the feature according to significance (Tan et al., 2005).

The employed five feature subset selection techniques in this research work, are Ada-Boost, CFS, Consistency-based, Logistic Regression Analysis, and Naïve Bayes. Ada-boost fits a classifier on a dataset and assigns weights to the classifier (Tan et al., 2005), based on this, it finds a set of best weak classifiers for given training data. CFS gives a metric to evaluate the efficacy of a feature subset. This method of feature selection chooses a subset of features that are greatly correlated with the class (Hall et al., 1999). The consistency-based method focuses more on inconsistency measures of the feature subset instead of the goodness of the feature subset. Univariate logistic regression is used to determine the importance of attributes. The model selects the features based on their p-values. Features having a  $p$ -value  $\leq 0.05$  are considered to be of more relevance. Naïve Bayes is a supervised algorithm and is based on Bayes (Tan et al., 2005).

#### 3.3. Sampling techniques

The class imbalance issue in input datasets can be tackled by re-sampling the original dataset, either by over or under-sampling the minority and/or majority classes. In this study to accomplish the greater precision power of the SFP model by fixing the data imbalance problem, we have used eight different sampling techniques listed below in Table 6.

#### 3.4. Classification techniques

The major objective of this study is to assess the impact of class imbalance and class irrelevance on the performance of SFP models. And analyze the better prediction power post implication of stated different sampling and feature selection techniques. To train the SPF model we have employed eight popular classifiers (Pandey et al., 2021), shown in Table 7.

<sup>1</sup> <https://github.com/sonikacrathi/OODatasets.git>.

**Table 2**  
Object-oriented software project datasets.

| PR_ID | # Faulty_class | # class | PR_Name       |
|-------|----------------|---------|---------------|
| 1     | 32             | 293     | ant-1.5       |
| 2     | 16             | 157     | synapse-1.0   |
| 3     | 77             | 162     | xerces-init   |
| 4     | 27             | 234     | arc           |
| 5     | 40             | 352     | ivy-2.0       |
| 6     | 60             | 222     | synapse-1.1   |
| 7     | 14             | 25      | szybkafucha   |
| 8     | 40             | 178     | ant-1.4       |
| 9     | 92             | 351     | ant-1.6       |
| 10    | 203            | 340     | lucene-2.4    |
| 11    | 86             | 256     | synapse-1.2   |
| 12    | 9              | 45      | serapion      |
| 13    | 166            | 745     | ant-1.7       |
| 14    | 20             | 125     | ant-1.3       |
| 15    | 6              | 27      | kalkulator    |
| 16    | 9              | 65      | systemdata    |
| 17    | 9              | 45      | skarbonka     |
| 18    | 91             | 195     | lucene-2.0    |
| 19    | 10             | 27      | nieruchomosci |
| 20    | 144            | 247     | lucene-2.2    |
| 21    | 20             | 39      | workflow      |
| 22    | 13             | 339     | camel-1.0     |
| 23    | 77             | 858     | tomcat        |
| 24    | 27             | 176     | redaktor      |
| 25    | 63             | 111     | ivy-1.1       |
| 26    | 15             | 33      | pdftranslator |
| 27    | 71             | 440     | xerces-1.2    |
| 28    | 69             | 453     | xerces-1.3    |
| 29    | 216            | 608     | camel-1.2     |
| 30    | 78             | 229     | velocity-1.6  |
| 31    | 188            | 965     | camel-1.6     |
| 32    | 142            | 214     | velocity-1.5  |
| 33    | 437            | 588     | xerces-1.4    |
| 34    | 145            | 872     | camel-1.4     |
| 35    | 16             | 241     | ivy-1.4       |
| 36    | 13             | 29      | zuzel         |
| 37    | 840            | 8718    | prop-4        |
| 38    | 1299           | 8516    | prop-5        |
| 39    | 66             | 660     | prop-6        |
| 40    | 12             | 18      | wspomaganicpi |
| 41    | 11             | 492     | jedit-4.3     |
| 42    | 2431           | 23 014  | prop-2        |
| 43    | 1180           | 10 274  | prop-3        |
| 44    | 48             | 367     | jedit-4.2     |
| 45    | 75             | 306     | jedit-4.0     |
| 46    | 2738           | 18 471  | prop-1        |
| 47    | 79             | 312     | jedit-4.1     |
| 48    | 12             | 20      | sklebagd      |
| 49    | 16             | 43      | berek         |
| 50    | 13             | 42      | termoproject  |
| 51    | 90             | 272     | jedit-3.2     |
| 52    | 37             | 109     | log4j-1.1     |
| 53    | 34             | 135     | log4j-1.0     |
| 54    | 189            | 205     | log4j-1.2     |

### 3.5. Statistical tests

To evaluate the stated hypothesis during the result analysis statistically, we have used two techniques: The Wilcoxon Signed Rank test (Woolson, 2007) and the Friedman test (Zimmerman & Zumbo, 1993). These tests are non-parametric and are utilized to see if the performance of the different techniques in this study differed considerably when compared to one another. There is no necessity that the data to follow a normal distribution to use these strategies. The Wilcoxon Sign Rank Test (Woolson, 2007) is a statistical method for calculating significant differences between two related samples or paired instances to see if their mean value range diverges. The Friedman test (Zimmerman & Zumbo, 1993) is another strategy we considered in this study. This test assigns a rating to each technique based on the AUC value of each technique/algorithm as an evaluation criterion. The technique with the lowest rank is thought to be the most efficient.

**Table 3**  
Software metrics considered in this study.

| OO-metrics                                      | Metrics-explanation                                                                                                                                                                                            |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Afferent coupling (Ca)                          | The specific class is used by a number of other classes                                                                                                                                                        |
| Average McCabe's cyclomatic complexity (Avg-CC) | Average cyclomatic complexity of a class's methods                                                                                                                                                             |
| Average method complexity (AMC)                 | Each class's average method size                                                                                                                                                                               |
| Cohesion among methods of class (CAM)           | The sum of the number of different types of method parameters in each method divided by the multiplication of the number of different types of method parameters in the entire class and the number of methods |
| Coupling between methods (CBM)                  | The number of new methods to which all inherited methods are linked                                                                                                                                            |
| Coupling between object classes (CBO)           | Number of classes associated with a given class                                                                                                                                                                |
| Data access metric (DAM)                        | The ratio of private (protected) attributes in the class to the total number of declared attributes                                                                                                            |
| Depth of inheritance tree (DIT)                 | The class hierarchy's maximum height                                                                                                                                                                           |
| Efferent coupling (Ce)                          | The number of classes that a specific class uses                                                                                                                                                               |
| Inheritance coupling (IC)                       | The number of parent classes with which a given class is associated.                                                                                                                                           |
| Lack of cohesion in methods (LCOM)              | The number of methods in a class that are unrelated despite the fact that some of the class's fields are shared                                                                                                |
| LCOM3                                           | Methods lack of cohesion. Henderson-Sellers version                                                                                                                                                            |
| Lines of code (LOC)                             | The number of lines in the source code's text                                                                                                                                                                  |
| Max cyclomatic complexity (Max-CC)              | Maximum cyclomatic complexity of a class's methods                                                                                                                                                             |
| The measure of aggregation (MOA)                | Number of data declarations (class fields) with user-defined class types                                                                                                                                       |
| Measure of functional abstraction (MFA)         | The ratio of the number of methods inherited by a class to the total number of methods accessible by the class's member methods                                                                                |
| Number of children (NOC)                        | Number of immediate descendants of the class                                                                                                                                                                   |
| Number of public methods (NPM)                  | The number of public methods declared in a class                                                                                                                                                               |
| Response for a Class (RFC)                      | The number of different methods that can be executed when a message is received by an object of that class                                                                                                     |
| Weighted methods per class (WMC)                | Sum of the complexities of methods defined in class                                                                                                                                                            |

### 3.6. Performance parameters

This section gives details about the fundamental definition of performance parameters used in this study for comparing different techniques used for software fault prediction. A confusion matrix has been used for computing these parameters. The actual and predicted categorization information done by the defect prediction technique is maintained in this matrix. Table 8 shows the confusion matrix for this model.

In this research accuracy and AUC, evaluators have been used to measure the performance. The formula to calculate the accuracy is:

$$\frac{No_{NF \rightarrow NF} + No_{F \rightarrow F}}{No_{NF \rightarrow NF} + No_{NF \rightarrow F} + No_{F \rightarrow NF} + No_{F \rightarrow F}} = \frac{\text{correct labels}}{\text{number of samples}} \quad (1)$$

The AUC is a summary of the ROC curve that assesses the ability of a classifier to distinguish between classes. The AUC measures how successfully a model can distinguish between faulty and non-faulty

**Table 4**Descriptive statistics : 95% confidence interval:  $Mean(M) \pm 1.96 * \frac{STD(M)}{\sqrt{NOS}}$ .

| PR_ID | WMC        | CBO        | RFC         | LCOM          | Ca        | Ce         | LOC           | AMC         | MAX_cc     | AVG_cc    |
|-------|------------|------------|-------------|---------------|-----------|------------|---------------|-------------|------------|-----------|
| 1     | 10.1 ± 1.3 | 10.6 ± 2.1 | 30.7 ± 3.6  | 74.0 ± 35.5   | 5.6 ± 2.0 | 5.4 ± 0.6  | 297.1 ± 51.1  | 31.3 ± 13.8 | 4.6 ± 0.7  | 1.4 ± 0.1 |
| 2     | 8.1 ± 1.3  | 13.2 ± 1.6 | 29.8 ± 3.8  | 39.0 ± 22.2   | 4.3 ± 1.6 | 9.1 ± 1.0  | 183.5 ± 26.9  | 22.8 ± 3.0  | 6.5 ± 1.1  | 1.7 ± 0.2 |
| 3     | 11.4 ± 2.9 | 6.1 ± 1.2  | 23.3 ± 6.6  | 139.5 ± 113.1 | 3.3 ± 0.9 | 3.4 ± 0.9  | 560.0 ± 205.6 | 34.9 ± 13.5 | 3.5 ± 0.8  | 1.3 ± 0.2 |
| 4     | 9.2 ± 1.2  | 7.6 ± 1.1  | 21.3 ± 3.4  | 56.2 ± 17.8   | 2.9 ± 0.8 | 4.7 ± 0.8  | 133.9 ± 31.8  | 9.7 ± 1.3   | 2.8 ± 0.5  | 1.0 ± 0.1 |
| 5     | 11.3 ± 1.6 | 13.2 ± 1.7 | 34.0 ± 4.7  | 131.6 ± 74.4  | 6.9 ± 1.5 | 5.2 ± 0.9  | 249.3 ± 44.7  | 18.5 ± 2.8  | 3.2 ± 0.4  | 1.2 ± 0.1 |
| 6     | 7.3 ± 1.1  | 12.5 ± 1.4 | 28.6 ± 3.2  | 34.8 ± 17.2   | 4.1 ± 1.3 | 8.6 ± 0.9  | 190.5 ± 26.4  | 27.2 ± 3.3  | 7.1 ± 1.0  | 2.1 ± 0.2 |
| 7     | 6.8 ± 1.9  | 7.4 ± 1.2  | 17.2 ± 5.1  | 24.0 ± 21.7   | 2.2 ± 1.2 | 5.2 ± 1.6  | 76.4 ± 27.7   | 13.2 ± 6.2  | 2.9 ± 1.8  | 1.4 ± 0.5 |
| 8     | 10.5 ± 1.6 | 10.8 ± 2.4 | 33.8 ± 4.5  | 74.1 ± 40.5   | 5.7 ± 2.4 | 5.4 ± 0.7  | 304.5 ± 53.1  | 26.7 ± 3.6  | 4.3 ± 0.7  | 1.4 ± 0.1 |
| 9     | 11.1 ± 1.3 | 11.5 ± 2.2 | 34.2 ± 3.6  | 87.0 ± 35.8   | 6.1 ± 2.1 | 5.8 ± 0.6  | 322.6 ± 47.8  | 30.1 ± 11.6 | 5.1 ± 0.7  | 1.5 ± 0.1 |
| 10    | 10.4 ± 1.4 | 10.8 ± 1.3 | 25.2 ± 3.4  | 68.7 ± 47.1   | 6.0 ± 1.2 | 5.4 ± 0.7  | 302.5 ± 68.0  | 23.0 ± 2.9  | 4.2 ± 0.9  | 1.3 ± 0.1 |
| 11    | 7.6 ± 1.1  | 12.8 ± 1.4 | 29.5 ± 3.2  | 41.1 ± 21.5   | 4.3 ± 1.2 | 8.7 ± 0.8  | 209.0 ± 28.6  | 28.2 ± 3.3  | 7.8 ± 1.1  | 2.1 ± 0.2 |
| 12    | 8.2 ± 2.2  | 6.5 ± 1.4  | 28.8 ± 7.8  | 37.4 ± 21.7   | 3.2 ± 1.2 | 4.0 ± 1.2  | 334.0 ± 121.0 | 35.9 ± 12.0 | 3.5 ± 1.7  | 1.1 ± 0.3 |
| 13    | 11.1 ± 0.9 | 11.0 ± 1.9 | 34.4 ± 2.6  | 89.1 ± 25.1   | 5.7 ± 1.9 | 5.7 ± 0.4  | 280.1 ± 29.6  | 23.6 ± 5.5  | 4.7 ± 0.5  | 1.4 ± 0.1 |
| 14    | 10.6 ± 1.8 | 10.4 ± 2.6 | 34.4 ± 5.1  | 69.3 ± 45.5   | 5.6 ± 2.6 | 5.2 ± 0.8  | 301.6 ± 59.4  | 27.1 ± 3.7  | 4.2 ± 0.9  | 1.4 ± 0.2 |
| 15    | 10.0 ± 6.8 | 6.7 ± 2.5  | 21.6 ± 7.1  | 192.9 ± 214.5 | 2.3 ± 0.8 | 4.9 ± 2.3  | 149.0 ± 50.0  | 33.7 ± 11.9 | 2.6 ± 2.8  | 0.8 ± 0.5 |
| 16    | 9.0 ± 2.2  | 7.1 ± 1.7  | 20.5 ± 5.0  | 55.0 ± 38.9   | 4.4 ± 1.4 | 3.6 ± 1.0  | 237.6 ± 104.3 | 21.4 ± 7.0  | 2.4 ± 0.7  | 1.0 ± 0.1 |
| 17    | 6.6 ± 1.6  | 7.2 ± 1.3  | 28.0 ± 6.9  | 18.2 ± 12.2   | 3.5 ± 1.4 | 4.2 ± 0.8  | 233.4 ± 109.3 | 29.5 ± 9.3  | 2.0 ± 0.8  | 0.9 ± 0.2 |
| 18    | 9.3 ± 1.2  | 9.8 ± 1.4  | 23.1 ± 3.1  | 35.2 ± 20.2   | 5.5 ± 1.3 | 4.7 ± 0.7  | 259.5 ± 61.6  | 23.2 ± 3.3  | 5.3 ± 2.6  | 1.3 ± 0.2 |
| 19    | 4.5 ± 1.2  | 4.0 ± 1.0  | 17.1 ± 4.8  | 3.5 ± 2.0     | 1.9 ± 0.6 | 2.1 ± 0.8  | 176.1 ± 64.5  | 41.7 ± 21.7 | 3.3 ± 1.7  | 1.3 ± 0.4 |
| 20    | 9.4 ± 1.2  | 10.0 ± 1.5 | 22.9 ± 3.0  | 39.8 ± 22.4   | 5.7 ± 1.3 | 4.7 ± 0.7  | 257.4 ± 57.2  | 22.0 ± 2.8  | 4.9 ± 2.1  | 1.3 ± 0.2 |
| 21    | 12.8 ± 4.7 | 7.2 ± 1.5  | 23.3 ± 9.1  | 142.3 ± 165.7 | 3.4 ± 1.4 | 3.9 ± 1.0  | 105.8 ± 48.4  | 6.0 ± 1.1   | 1.7 ± 0.5  | 0.9 ± 0.1 |
| 22    | 8.1 ± 0.9  | 10.0 ± 1.6 | 19.6 ± 2.1  | 53.7 ± 22.1   | 5.0 ± 1.6 | 5.7 ± 0.6  | 99.5 ± 13.8   | 10.4 ± 1.0  | 2.0 ± 0.3  | 0.9 ± 0.1 |
| 23    | 13.0 ± 1.2 | 7.6 ± 0.7  | 33.5 ± 3.0  | 176.3 ± 77.6  | 3.9 ± 0.6 | 0.0 ± 0.0  | 350.4 ± 43.1  | 25.6 ± 3.1  | 4.3 ± 0.5  | 1.3 ± 0.1 |
| 24    | 10.1 ± 1.0 | 12.1 ± 1.6 | 27.4 ± 3.2  | 6.5 ± 6.5     | 5.6 ± 1.5 | 6.5 ± 0.9  | 336.8 ± 53.2  | 29.2 ± 2.6  | 3.7 ± 0.4  | 1.4 ± 0.1 |
| 25    | 10.6 ± 2.2 | 10.4 ± 1.8 | 32.4 ± 7.7  | 87.1 ± 62.5   | 5.3 ± 1.4 | 5.7 ± 1.4  | 245.9 ± 90.5  | 18.2 ± 4.4  | 3.1 ± 0.7  | 1.2 ± 0.1 |
| 26    | 7.7 ± 2.9  | 6.3 ± 2.1  | 27.2 ± 10.3 | 44.1 ± 42.6   | 2.3 ± 0.8 | 4.4 ± 1.9  | 191.5 ± 115.4 | 19.8 ± 7.0  | 2.6 ± 1.3  | 1.1 ± 0.3 |
| 27    | 11.3 ± 1.2 | 4.9 ± 0.8  | 21.2 ± 2.9  | 91.0 ± 21.8   | 2.5 ± 0.6 | 2.7 ± 0.4  | 361.9 ± 94.9  | 21.7 ± 5.5  | 3.4 ± 0.8  | 1.2 ± 0.1 |
| 28    | 11.4 ± 1.2 | 5.1 ± 0.8  | 21.7 ± 3.0  | 94.5 ± 23.0   | 2.7 ± 0.6 | 2.8 ± 0.4  | 368.9 ± 97.5  | 22.0 ± 5.4  | 3.5 ± 0.8  | 1.2 ± 0.1 |
| 29    | 8.3 ± 0.8  | 10.1 ± 1.4 | 20.2 ± 1.7  | 61.2 ± 21.2   | 5.0 ± 1.4 | 5.6 ± 0.5  | 109.0 ± 11.5  | 11.0 ± 1.0  | 2.1 ± 0.2  | 0.9 ± 0.0 |
| 30    | 9.0 ± 1.8  | 10.8 ± 1.6 | 23.0 ± 3.5  | 80.3 ± 71.9   | 5.6 ± 1.4 | 6.0 ± 1.0  | 249.0 ± 133.9 | 19.6 ± 3.6  | 4.0 ± 1.9  | 1.3 ± 0.2 |
| 31    | 8.6 ± 0.7  | 11.1 ± 1.4 | 21.4 ± 1.6  | 79.3 ± 33.0   | 5.3 ± 1.4 | 6.4 ± 0.4  | 117.2 ± 11.1  | 11.1 ± 0.8  | 2.3 ± 0.2  | 1.0 ± 0.0 |
| 32    | 9.0 ± 1.9  | 10.6 ± 1.6 | 22.6 ± 3.6  | 78.8 ± 74.2   | 5.6 ± 1.4 | 5.9 ± 1.0  | 248.3 ± 136.4 | 19.8 ± 3.7  | 3.9 ± 1.9  | 1.2 ± 0.2 |
| 33    | 9.9 ± 0.9  | 6.3 ± 0.7  | 19.2 ± 2.1  | 75.5 ± 17.3   | 3.3 ± 0.6 | 3.3 ± 0.4  | 240.1 ± 48.1  | 17.5 ± 2.7  | 3.8 ± 0.8  | 1.4 ± 0.1 |
| 34    | 8.5 ± 0.7  | 10.8 ± 1.4 | 21.2 ± 1.6  | 73.4 ± 28.5   | 5.1 ± 1.3 | 6.3 ± 0.5  | 112.5 ± 10.8  | 10.9 ± 0.8  | 2.2 ± 0.2  | 0.9 ± 0.0 |
| 35    | 10.7 ± 2.1 | 11.3 ± 1.8 | 31.3 ± 6.2  | 163.2 ± 169.6 | 5.8 ± 1.5 | 4.1 ± 1.0  | 246.0 ± 80.2  | 18.2 ± 3.8  | 3.1 ± 0.5  | 1.2 ± 0.1 |
| 36    | 13.9 ± 4.0 | 7.5 ± 3.1  | 35.0 ± 9.9  | 85.1 ± 52.6   | 5.6 ± 2.9 | 3.0 ± 1.1  | 497.3 ± 225.4 | 26.3 ± 8.7  | 2.5 ± 1.1  | 1.2 ± 0.3 |
| 37    | 4.3 ± 0.1  | 14.4 ± 0.5 | 24.0 ± 0.6  | 21.6 ± 4.2    | 2.4 ± 0.4 | 12.0 ± 0.2 | 173.0 ± 7.6   | 33.4 ± 0.8  | 2.8 ± 0.1  | 1.1 ± 0.0 |
| 38    | 4.0 ± 0.1  | 16.9 ± 0.5 | 21.3 ± 0.4  | 13.8 ± 4.5    | 2.7 ± 0.5 | 14.1 ± 0.2 | 127.0 ± 4.5   | 32.9 ± 1.1  | 2.3 ± 0.1  | 1.0 ± 0.0 |
| 39    | 8.7 ± 0.6  | 10.2 ± 0.7 | 24.2 ± 1.7  | 41.7 ± 6.5    | 4.5 ± 0.6 | 6.0 ± 0.5  | 147.8 ± 12.2  | 17.4 ± 1.6  | 3.4 ± 0.3  | 1.3 ± 0.1 |
| 40    | 7.7 ± 3.1  | 6.9 ± 3.0  | 42.8 ± 12.9 | 33.1 ± 41.5   | 3.8 ± 1.6 | 5.4 ± 3.1  | 315.8 ± 121.6 | 39.0 ± 12.1 | 2.3 ± 0.9  | 1.1 ± 0.2 |
| 41    | 12.4 ± 2.2 | 14.3 ± 2.2 | 39.8 ± 5.0  | 259.9 ± 193.0 | 8.7 ± 1.9 | 7.1 ± 0.8  | 411.3 ± 83.7  | 28.3 ± 3.0  | 6.6 ± 1.0  | 1.8 ± 0.2 |
| 42    | 5.2 ± 0.1  | 16.3 ± 0.3 | 24.8 ± 0.5  | 59.6 ± 14.2   | 2.7 ± 0.3 | 13.6 ± 0.1 | 162.9 ± 5.0   | 31.5 ± 0.5  | 3.1 ± 0.1  | 1.3 ± 0.0 |
| 43    | 4.5 ± 0.1  | 16.1 ± 0.3 | 23.8 ± 0.5  | 18.1 ± 3.8    | 2.2 ± 0.3 | 13.9 ± 0.2 | 156.2 ± 6.0   | 31.2 ± 0.8  | 3.1 ± 0.1  | 1.3 ± 0.0 |
| 44    | 13.2 ± 2.8 | 14.1 ± 2.2 | 41.0 ± 5.9  | 310.8 ± 254.9 | 8.6 ± 2.0 | 7.2 ± 0.9  | 465.1 ± 111.0 | 31.1 ± 3.7  | 7.3 ± 1.3  | 1.9 ± 0.2 |
| 45    | 12.9 ± 3.5 | 12.4 ± 2.0 | 38.2 ± 6.4  | 197.4 ± 136.8 | 7.5 ± 1.8 | 6.4 ± 0.9  | 473.2 ± 177.6 | 31.3 ± 4.2  | 6.2 ± 1.1  | 1.8 ± 0.2 |
| 46    | 6.5 ± 0.1  | 12.5 ± 0.2 | 26.8 ± 0.5  | 39.5 ± 3.3    | 3.5 ± 0.2 | 9.2 ± 0.2  | 206.6 ± 6.1   | 26.0 ± 0.5  | 4.4 ± 0.1  | 1.5 ± 0.0 |
| 47    | 13.1 ± 3.4 | 13.0 ± 2.1 | 39.9 ± 6.4  | 187.9 ± 120.4 | 7.9 ± 1.9 | 6.6 ± 0.8  | 490.7 ± 174.4 | 31.4 ± 4.1  | 7.1 ± 1.4  | 1.9 ± 0.2 |
| 48    | 16.5 ± 6.9 | 6.3 ± 1.7  | 42.3 ± 11.5 | 146.0 ± 159.6 | 3.9 ± 1.5 | 4.0 ± 1.7  | 480.1 ± 245.2 | 32.1 ± 10.2 | 6.9 ± 4.2  | 1.7 ± 1.0 |
| 49    | 18.7 ± 6.3 | 10.4 ± 3.4 | 57.9 ± 23.0 | 260.4 ± 190.0 | 6.6 ± 2.5 | 7.4 ± 3.6  | 751.6 ± 410.0 | 27.1 ± 6.0  | 11.4 ± 5.3 | 1.5 ± 0.3 |
| 50    | 4.8 ± 1.6  | 8.2 ± 1.8  | 20.1 ± 8.8  | 13.4 ± 16.5   | 4.7 ± 1.2 | 4.7 ± 1.9  | 196.2 ± 116.2 | 27.7 ± 8.4  | 1.8 ± 0.5  | 0.9 ± 0.2 |
| 51    | 12.5 ± 3.6 | 12.0 ± 2.0 | 37.7 ± 6.6  | 171.3 ± 113.6 | 7.2 ± 1.7 | 6.2 ± 0.8  | 473.8 ± 192.6 | 32.6 ± 4.6  | 6.3 ± 1.4  | 1.7 ± 0.2 |
| 52    | 7.8 ± 1.3  | 6.9 ± 1.5  | 23.2 ± 3.3  | 27.0 ± 13.3   | 3.5 ± 1.4 | 3.5 ± 0.6  | 182.9 ± 35.0  | 21.2 ± 3.5  | 3.8 ± 0.7  | 1.5 ± 0.2 |
| 53    | 6.6 ± 1.0  | 6.9 ± 1.5  | 21.4 ± 3.0  | 19.9 ± 11.0   | 3.7 ± 1.4 | 3.5 ± 0.5  | 159.6 ± 31.1  | 20.2 ± 3.0  | 3.3 ± 0.6  | 1.3 ± 0.1 |
| 54    | 8.4 ± 1.3  | 7.6 ± 1.3  | 25.2 ± 3.9  | 53.9 ± 47.8   | 4.3 ± 1.2 | 3.8 ± 0.7  | 186.3 ± 35.1  | 19.8 ± 2.5  | 3.3 ± 0.4  | 1.3 ± 0.1 |

**Table 5**

Naming conventions used for employed feature selection strategies.

| Abbreviation | Corresponding name                  |
|--------------|-------------------------------------|
| AM           | All metrics                         |
| FR1          | RELIEF                              |
| FR2          | Gain ratio feature evaluation       |
| FR3          | Information gain feature evaluation |
| FR4          | OneR feature evaluation             |
| FR5          | Chi-squared test                    |
| FS1          | Adaboost                            |
| FS2          | Correlation-based feature selection |
| FS3          | Consistency                         |
| FS4          | Logistic regression analysis        |
| FS5          | NaiveBayes                          |

classifications. The higher the AUC value, the better the performance of the prediction model.

#### 4. Proposed research framework and experimental analysis

The proposed framework and experimental results of utilized sampling and feature selection strategies, and the performance of 8 classifiers are presented in this section of the paper. This section also includes a review of the outcomes of SFP models with and without the use of the aforementioned sampling and feature selection strategies.

##### 4.1. Software fault prediction model framework

The proposed Software Fault Prediction model framework constitutes five steps as shown in Fig. 1.

- **Step 1:** In this step, faulty software metrics data was collected from 54 open-source object-oriented software projects. The class-level software feature metrics considered for the analysis are mentioned in Table 3 (Kumar et al., 2018).

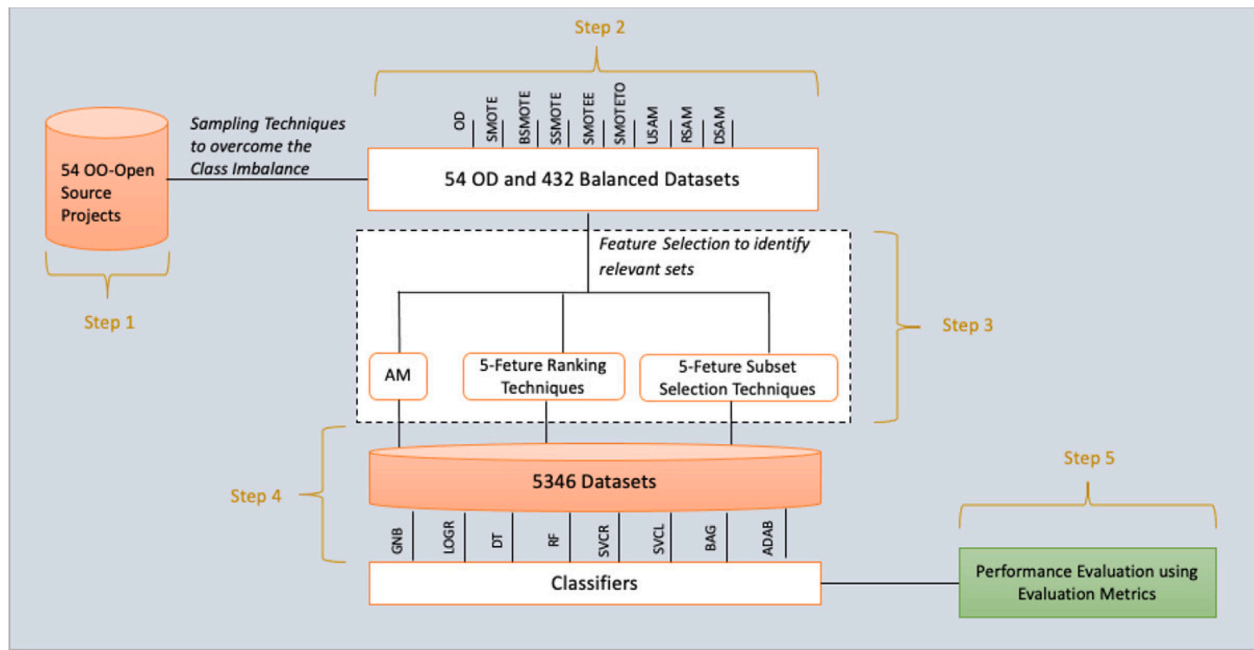


Fig. 1. Software fault prediction model framework.

Table 6

Sampling techniques used in this study.

| Employed sampling techniques                       | Description                                                                                                                                                         |
|----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synthetic Minority Over Sampling Technique (SMOTE) | Selects a minority class sample from a data set and then synthesize a new instance by choosing one of the k-nearest neighbors of the chosen minority class instance |
| Borderline SMOTE (BSMOTE)                          | Generates the synthetic instances of minority classes by choosing nearest neighbors that lie on the decision boundary between classes                               |
| SVM-SMOTE (SSMOTE)                                 | Uses the SVM algorithm to identify misclassification boundaries                                                                                                     |
| SMOTE- Edited (SMOTEE)                             | Combines oversampling and undersampling techniques for imbalanced classification                                                                                    |
| SMOTE with Tomek's link (SMOTETO)                  | First SMOTE is applied to oversample the minority class and then examples in Tomek Links from the majority classes are identified and removed                       |
| Upsampling (USAM)                                  | Duplicates record from minority classes and injects into the dataset to prevent the model from inclining towards the majority class                                 |
| Under Random sampling (RSAM)                       | Every element of the population has an equal probability of getting selected                                                                                        |
| Down sampling (DSAM)                               | Refers to training on a disproportionately low subset of majority class examples                                                                                    |

Table 7

Description of employed classifiers.

| Classifier                  | Description                                                                                                                                  |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Gaussian Naive Bayes (GNB)  | Variant of Naive Bayes that follows a normal distribution and supports continuous data                                                       |
| Logistic Regression (LOGR)  | Supervised learning technique used to predict the probability of a target variable                                                           |
| Decision tree (DT)          | Supervised learning, constructed in a manner that data is continuously split based on some parameter                                         |
| Random forest (RF)          | Ensemble learning consists of many decision trees. The forest generated by the algorithm is trained through bagging or bootstrap aggregation |
| SVC                         | Uses SVM with a radial basis function which is a kernel function to find linear classifier/regression boundary                               |
| SVCL                        | Uses SVM with a linear basis function which is a kernel function to find linear classifier/regression boundary                               |
| Bootstrap Aggregation (BAG) | Used to reduce the variation of calculations for a large variance                                                                            |
| AdaBoost (ADAB)             | Employs a lot of classifiers for increasing classifier accuracy. AdaBoost is a program that allows you to create iterative ensembles         |

Table 8

Confusion matrix.

|            | Non-faulty              | Faulty                 |
|------------|-------------------------|------------------------|
| Non-faulty | No. $NF \rightarrow NF$ | No. $NF \rightarrow F$ |
| Faulty     | No. $F \rightarrow NF$  | No. $F \rightarrow F$  |

- Step 2:** While reviewing and inspecting the datasets, we observed class imbalance. Henceforth, to handle the class imbalance problem and its impact on prediction accuracy the framework followed the oversampling and undersampling techniques during this step. In this model 8 different sampling techniques namely Synthetic Minority Over Sampling Technique (SMOTE), Borderline SMOTE (BSMOTE), SVM-SMOTE (SSMOTE), SMOTE-Edited (SMOTEE), USAM, SMOTE with Tomek's link (SMOTETO), RSAM, and DSAM has been applied which provided 432 balanced datasets as an outcome. To do the performance validation and analysis we did consider 54 original datasets along with derived balanced 486 variants of input datasets.
- Step 3:** In step 3, against the balanced datasets, we considered applying Feature Selection Techniques as illustrated in the above

framework, to identify relevant sets and minimize the impact of class irrelevance by removing correlated and redundant features. In this Feature Selection implementation, the proposed framework followed Feature Ranking and Feature Subset Selection methods. For the Feature Ranking method, we applied five techniques i.e. oneR, Relief, Gain Ratio, Info-Gain, and Chi-square to extract the top  $\log_2(N)$  features useful for the classification task whereas for Feature Subset Selection we applied Ada-Boost, CFS, Consistency, Logistic and Naive Bayes to develop a subset of best

features from each method. For comparative analysis and validating the predictive power of developed SFP models, we do consider relevant features with all feature metrics as well. This results in a total of 5346 distinct input dataset variations to analyze and measure the performance of the SFP models developed using 8 different classifiers.

- **Step 4:** During this step, we have applied the most commonly used state-of-the-art classifiers (Pandey et al., 2021) like Gaussian Naïve Bayes (GNB), Logistic Regression (LOGR), Decision Tree (DT), Random Forest (RF), SVCR, SVCL, Bagging (BAG), and AdaBoost (ADAB) to predict the faulty class in the datasets. This step resulted in 8 different SFP models with combinations of 11 feature selections (1 for all metrics) and 9 sampling methods (1 unsampled dataset) against 54 open-source datasets.
- **Step 5:** To perform a comparative analysis and to evaluate the performance and impact of various sampling techniques and feature selection methods over fault predictability. In this step, the performance and reliability of these models were evaluated using the performance parameters: accuracy and AUC. Additionally, we used two statistical tests the Wilcoxon Signed Rank and the Friedman Test, to evaluate the effectiveness of each of the employed feature selection and sampling methods separately as well as when combined. This step involved performing a comparative analysis to validate the outcome of 8 state-of-the-art classifiers with/without the application of feature selection and sampling approaches.

#### 4.2. Results and analysis of combined approaches

In this segment of the paper, we showcase the results & performance obtained from feature ranking and feature subset selection techniques over class-level metrics on the imbalanced as well as on the balanced dataset generated from sampling techniques. To get these balanced datasets we first used the stated 8 different sampling techniques to overcome the class imbalance issue. Then we employed 8 different classifiers to detect the faulty class and evaluate the effectiveness of the implication of sampling and feature selection techniques for SFP models. The model's effectiveness was computed using different performance parameters. Considering the space constraint, we have included the results of the one randomly selected feature ranking technique and feature subset selection technique.

##### *How powerful are the various feature selection methods in terms of optimizing the performance of the SFP model ?*

Here, in this study, we would like to compare and contrast feature-subset selection and feature ranking techniques post-application of mentioned sampling techniques to examine if any of the techniques is superior to the others or if all the techniques perform equally well.

##### 4.2.1. Feature ranking

In Feature ranking techniques, the ranking of the features is performed independently without using any learning system. Feature ranking involves scoring each feature using a certain method and then choosing the best features based on their results. In this study, we applied five Feature Ranking techniques i.e. oneR, Relief, Gain Ratio, Info-Gain, and Chi-square to extract the top  $\log_2(N)$  features useful for the classification task. The outcome of the Relief feature ranking technique applied to the generated balanced dataset using different sampling techniques is illustrated in Fig. 2. For better presentation & understanding, we have chosen 4 sampling techniques namely SMOTE, BSMOTE, SSMOTE, and SMOTEE.

In Fig. 2; the X-axis represents the project's datasets (1 to 54) and Y-axis represents the class-level metrics. The metrics selected post feature ranking technique over SMOTE balanced datasets are represented by a circle shape, and the square symbol shows the metrics selected over BSMOTE balanced datasets. The symbol star & dot shows the metrics

chosen by the feature ranking technique from a balanced dataset generated using SSMOTE and SMOTEE respectively. As demonstrated in Fig. 2, for project 1 (PR-1) the five metrics selected post feature ranking technique over balanced data using SMOTE are RFC, CE, LCOM3, DAM, and MFC. Whereas DIT, RFC, LCOM3, DAM, and MFC metrics were selected from PR-1 balanced datasets generated by BSMOTE.

The results of feature ranking selection techniques on balanced datasets generated using SMOTE, BSMOTE, SSMOTE and SMOTEE in Fig. 2 are represented by:

- **Circle:** source code metrics selected post SMOTE;
- **Square:** source code metrics selected post BSMOTE;
- **Star:** source code metrics selected post SSMOTE;
- **Dot:** source code metrics selected post SMOTEE

Similar analyses have been performed for the remaining 4 feature ranking techniques over a balanced dataset obtained by 8 sampling techniques as well as over original imbalanced datasets. Fig. 3; represents the Metrics Cloud post application of respective feature ranking methods. The most important and relevant metrics are shown in bold and big text in Fig. 3, while the less important and irrelevant metrics are shown in lite and small text. RFC, CBO, and LOC are a few of the most important metrics retrieved by the Gain ratio, Info-Gain, oneR, and Chi-square feature ranking methods. Avg-CC, NOC, and LCOM3 are a few of the less important metrics returned by the respective feature ranking methods.

##### 4.2.2. Feature subset selection

In most ML tasks, selecting feature subsets is a critical pre-processing step. The purpose of feature selection is to extract a minimal set of features that capture the essence and pattern of data and discard the irrelevant features. In this study, we applied five feature subset selection techniques which are AdaBoost, CFS, Consistency, Logistic and Naive Bayes. The outcome of the AdaBoost feature subset technique applied to the dataset sampled using different sampling techniques is illustrated in Fig. 4. For well understanding and presentation purposes, we have chosen 4 sampling techniques namely SMOTE, BSMOTE, SSMOTE, and SMOTEE.

In Fig. 4; the X-axis represents the project's datasets (1 to 54) and Y-axis represents the class-level metrics. We can refer from Fig. 4, that the two class-level metrics selected are "CE, and Max-CC" after applying the feature subset selection technique over balanced datasets of project 1 (PR-1) generated using SMOTE. And "DIT, RFC, IC, and Max-CC" metrics were selected from PR-1 balanced datasets generated by BSMOTE. Similarly, in balanced datasets using SSMOTE for PR-1, the chosen metrics post feature subset selection are "DIT, RFC, CE, MOA, MFA, IC, and CBM". And "LCOM3, LOC, MFA, and Avg-CC" class-level metrics were selected from PR-1 balanced datasets generated by SMOTEE.

The results of selected metrics post employing feature subset selection techniques on balanced datasets generated using SMOTE, BSMOTE, SSMOTE and SMOTEE for all projects are represented as:

- **Circle:** source code metrics selected post SMOTE;
- **Square:** source code metrics selected post BSMOTE;
- **Star:** source code metrics selected post SSMOTE;
- **Dot:** source code metrics selected post SMOTEE

Similar analyses have been performed for other 4 feature subset selection methods over a balanced dataset obtained by 8 sampling techniques as well as on original imbalanced datasets. Fig. 5; shows the Metrics Cloud after applying the appropriate feature subset techniques. The most important and relevant metrics are highlighted in bold and large text, while the less important and irrelevant metrics are highlighted in lite and small text in Fig. 5. The RFC, CBO, CE, and LOC are the most important metrics returned by the CFS and Consistency feature subset selection method. Where few of the important metrics

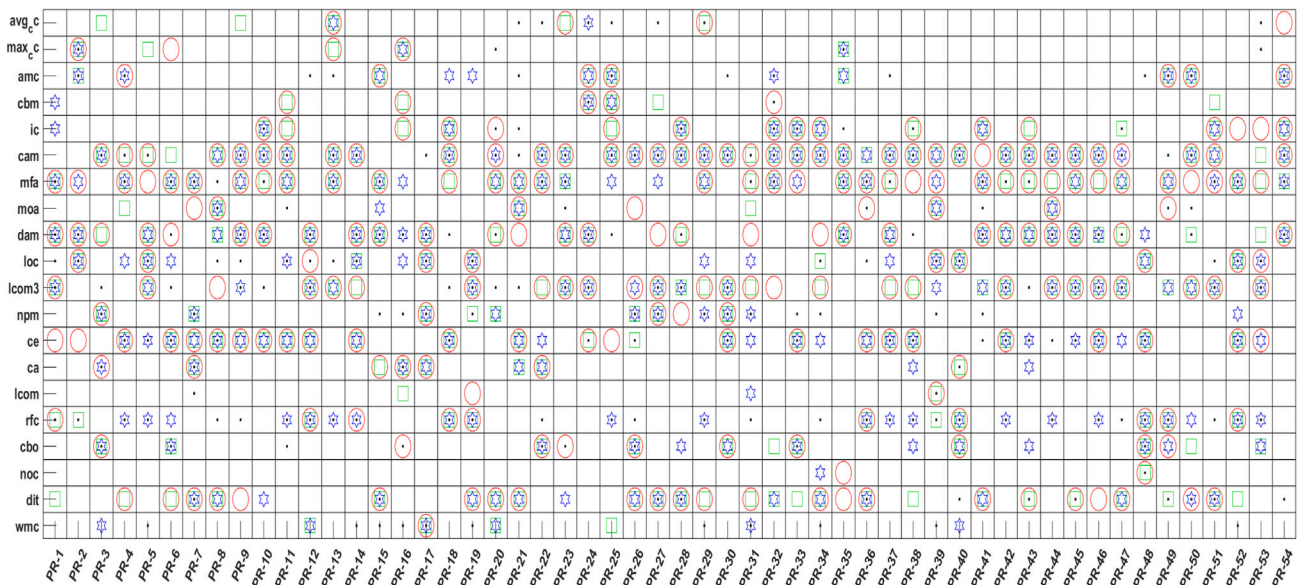


Fig. 2. Feature selected using relief.

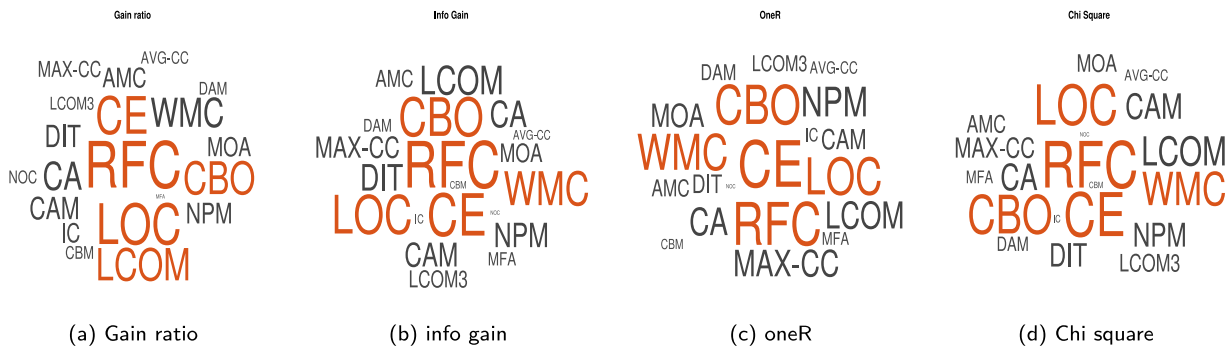


Fig. 3. Feature ranking: Metrics cloud.

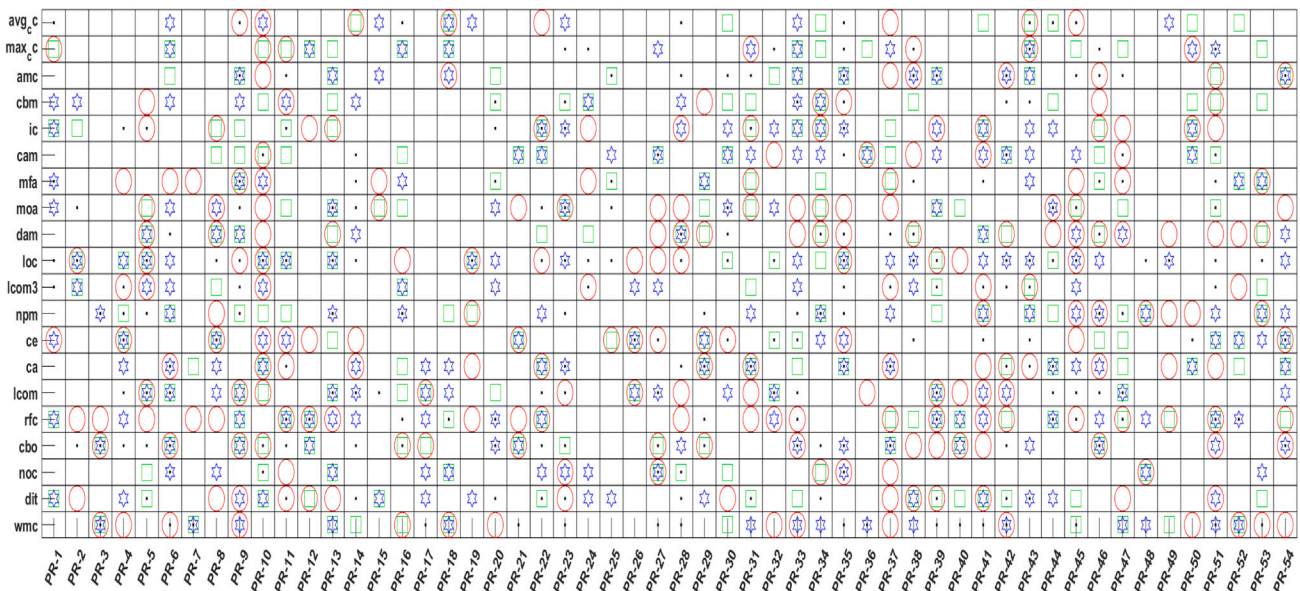


Fig. 4. Feature selected using AdaBoost.

retrieved by Logistic are RFC, CBO, NOC etc., and using Naïve Bayes are: CAM, LCOM3, DIT, etc., Some of the less important metrics marked

by feature subset selection strategies are DAM, IC, NPM, MOA, and so on.

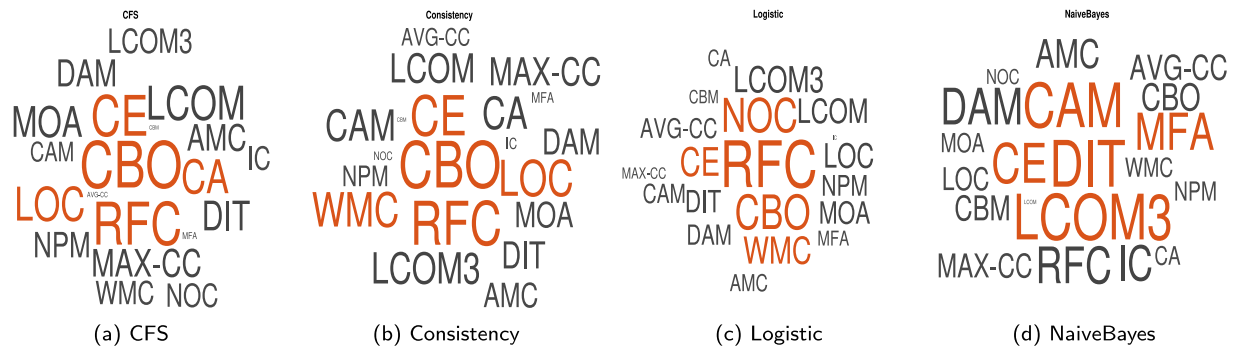


Fig. 5. Feature subset selection: Metrics cloud.

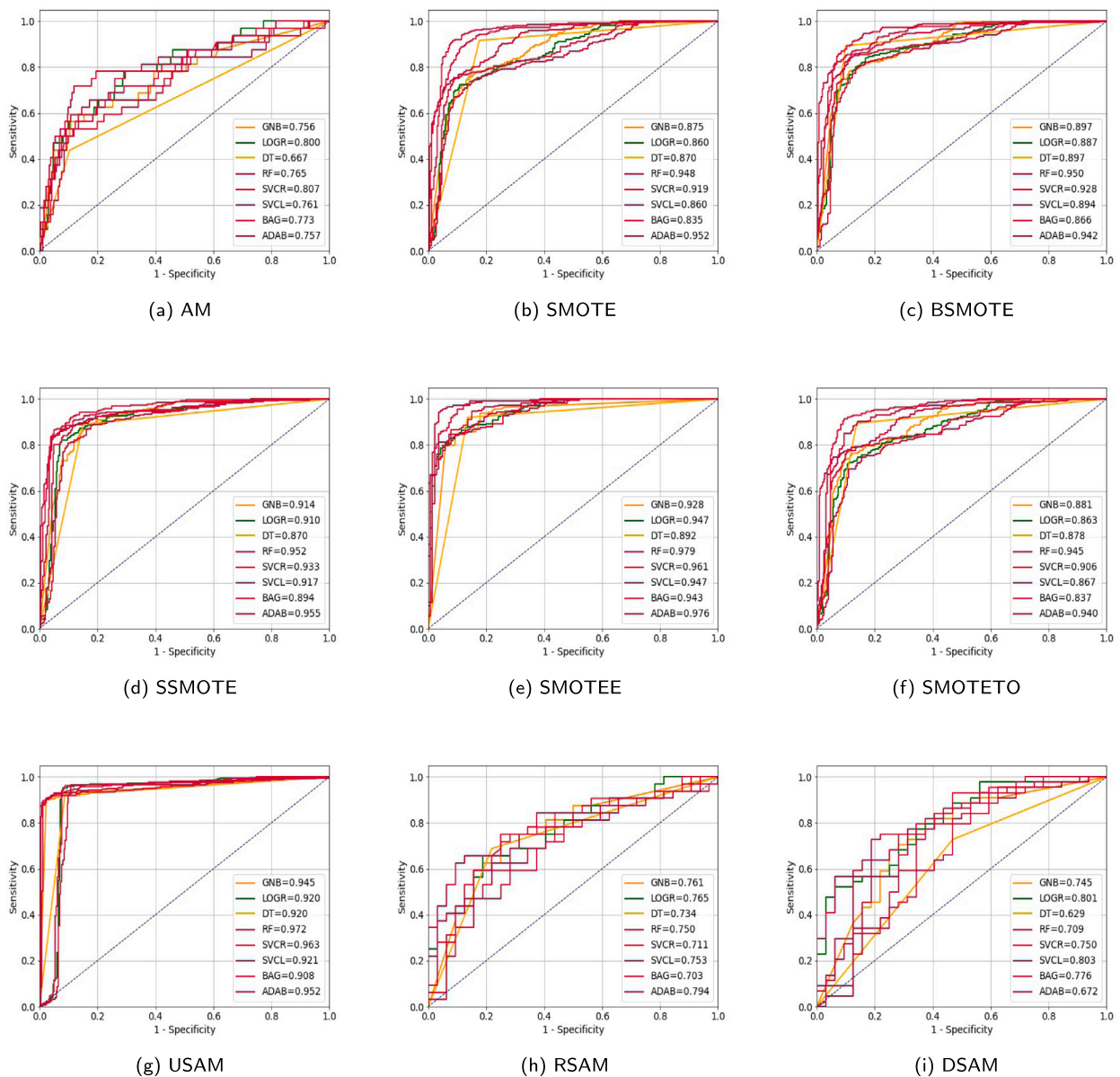


Fig. 6. AUC: Ant 1.5.

**Table 9**  
Accuracy and AUC for Ant 1.5.

|                  | Accuracy      |               |               |               |               |               |               |               | AUC         |             |      |             |             |             |             |             |
|------------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|-------------|-------------|------|-------------|-------------|-------------|-------------|-------------|
|                  | GNB           | LOGR          | DT            | RF            | SVCR          | SVCL          | BAG           | ADAB          | GNB         | LOGR        | DT   | RF          | SVCR        | SVCL        | BAG         | ADAB        |
| All Metrics (AM) |               |               |               |               |               |               |               |               |             |             |      |             |             |             |             |             |
| OD               | 74.40         | <b>89.761</b> | 84.64         | 88.74         | 89.08         | 89.08         | 88.74         | 88.74         | 0.76        | 0.8         | 0.67 | 0.77        | <b>0.81</b> | 0.76        | 0.77        | 0.76        |
| SMOTE            | 76.82         | 77.97         | 86.97         | 88.51         | 82.76         | 79.50         | 77.20         | <b>90.613</b> | 0.88        | 0.86        | 0.87 | <b>0.95</b> | 0.92        | 0.86        | 0.84        | <b>0.95</b> |
| BSMOTE           | 78.74         | 82.76         | <b>89.655</b> | 87.17         | 85.82         | 85.63         | 79.89         | 89.08         | 0.9         | 0.89        | 0.9  | <b>0.95</b> | 0.93        | 0.89        | 0.87        | 0.94        |
| SSMOTE           | 84.10         | 85.63         | 86.97         | 89.08         | 88.51         | 87.93         | 83.53         | <b>89.847</b> | 0.91        | 0.91        | 0.87 | 0.95        | 0.93        | 0.92        | 0.89        | <b>0.96</b> |
| SMOTEE           | 85.44         | 85.44         | 89.84         | 92.58         | 89.01         | 87.36         | 84.89         | <b>94.78</b>  | 0.93        | 0.95        | 0.89 | <b>0.98</b> | 0.96        | 0.95        | 0.94        | <b>0.98</b> |
| SMOTETO          | 76.77         | 78.15         | 87.80         | 87.60         | 83.27         | 80.32         | 77.36         | <b>90.748</b> | 0.88        | 0.86        | 0.88 | <b>0.94</b> | 0.91        | 0.87        | 0.84        | <b>0.94</b> |
| USAM             | 91.19         | 89.08         | 91.95         | <b>93.678</b> | 93.30         | 90.23         | 85.82         | 92.15         | 0.95        | 0.92        | 0.92 | <b>0.97</b> | 0.96        | 0.92        | 0.91        | 0.95        |
| RSAM             | 71.88         | 65.63         | <b>73.438</b> | 71.88         | 65.63         | 71.88         | 65.63         | 71.88         | 0.76        | 0.76        | 0.73 | 0.75        | 0.71        | 0.75        | 0.7         | <b>0.79</b> |
| DSAM             | 71.05         | 71.05         | 64.47         | 71.05         | 73.68         | 67.11         | <b>76.316</b> | 61.84         | 0.75        | <b>0.8</b>  | 0.63 | 0.71        | 0.75        | <b>0.8</b>  | 0.78        | 0.67        |
| Relief (FR1)     |               |               |               |               |               |               |               |               |             |             |      |             |             |             |             |             |
| OD               | 72.01         | 88.74         | 85.67         | <b>89.42</b>  | 88.06         | 89.08         | 89.08         | 89.08         | 0.66        | <b>0.8</b>  | 0.65 | 0.73        | 0.71        | 0.78        | 0.74        | 0.66        |
| SMOTE            | 57.66         | 79.12         | 89.08         | <b>90.613</b> | 78.74         | 76.44         | 83.72         | 88.89         | 0.81        | 0.85        | 0.89 | <b>0.96</b> | 0.87        | 0.85        | 0.92        | 0.94        |
| BSMOTE           | 64.56         | 81.80         | 88.51         | <b>94.061</b> | 80.27         | 79.12         | 85.63         | 90.23         | 0.83        | 0.88        | 0.89 | <b>0.97</b> | 0.89        | 0.88        | 0.92        | 0.94        |
| SSMOTE           | 68.39         | 84.48         | 89.08         | <b>92.72</b>  | 85.44         | 84.67         | 86.21         | 88.31         | 0.85        | 0.9         | 0.89 | <b>0.97</b> | 0.92        | 0.9         | 0.94        | 0.94        |
| SMOTEE           | 81.32         | 84.89         | 92.86         | <b>95.055</b> | 84.89         | 85.44         | 90.66         | 94.23         | 0.91        | 0.93        | 0.92 | <b>0.99</b> | 0.94        | 0.93        | 0.97        | 0.98        |
| SMOTETO          | 58.47         | 79.72         | 89.96         | <b>91.732</b> | 79.92         | 77.36         | 85.83         | 89.76         | 0.81        | 0.86        | 0.9  | <b>0.97</b> | 0.88        | 0.86        | 0.92        | 0.93        |
| USAM             | 88.51         | 90.42         | 89.85         | <b>94.253</b> | 92.15         | 90.61         | 92.15         | 93.87         | 0.96        | 0.93        | 0.9  | <b>0.97</b> | 0.95        | 0.93        | <b>0.97</b> | 0.96        |
| RSAM             | <b>64.063</b> | 59.38         | 56.25         | <b>64.063</b> | 59.38         | 60.94         | 60.94         | 59.38         | 0.71        | 0.62        | 0.56 | 0.68        | 0.66        | 0.76        | 0.66        | <b>0.77</b> |
| DSAM             | 64.47         | 69.74         | 69.74         | 73.68         | 73.68         | 69.74         | <b>75</b>     | 68.42         | 0.6         | 0.7         | 0.69 | 0.71        | <b>0.75</b> | 0.7         | 0.72        | 0.69        |
| Gain ratio (FR2) |               |               |               |               |               |               |               |               |             |             |      |             |             |             |             |             |
| OD               | 70.31         | 89.42         | 82.94         | <b>90.102</b> | 89.08         | 89.08         | 89.08         | 88.40         | 0.77        | <b>0.82</b> | 0.59 | 0.75        | 0.75        | 0.77        | 0.74        | 0.69        |
| SMOTE            | 59.39         | 78.93         | 90.04         | <b>90.996</b> | 77.78         | 77.20         | 83.33         | 89.66         | 0.79        | 0.85        | 0.9  | <b>0.97</b> | 0.87        | 0.85        | 0.91        | 0.93        |
| BSMOTE           | 63.60         | 81.80         | 90.61         | <b>91.379</b> | 80.84         | 79.69         | 86.97         | 88.51         | 0.8         | 0.87        | 0.91 | <b>0.96</b> | 0.88        | 0.87        | 0.92        | 0.94        |
| SSMOTE           | 70.12         | 85.06         | 90.81         | <b>91.188</b> | 85.63         | 84.10         | 86.40         | 90.23         | 0.87        | 0.9         | 0.91 | <b>0.96</b> | 0.92        | 0.91        | 0.94        | 0.95        |
| SMOTEE           | 81.87         | 84.62         | 93.13         | 93.68         | 85.44         | 85.44         | 88.74         | <b>94.231</b> | 0.9         | 0.93        | 0.92 | <b>0.98</b> | 0.94        | 0.93        | 0.96        | <b>0.98</b> |
| SMOTETO          | 50.79         | 62.40         | 85.24         | <b>86.221</b> | 64.76         | 61.81         | 83.47         | 84.25         | 0.72        | 0.67        | 0.85 | <b>0.94</b> | 0.77        | 0.67        | 0.9         | 0.9         |
| USAM             | 88.51         | 91.19         | 92.53         | <b>94.253</b> | 91.95         | 90.42         | 92.34         | 93.49         | 0.95        | 0.93        | 0.93 | <b>0.97</b> | 0.95        | 0.93        | <b>0.97</b> | 0.96        |
| RSAM             | 59.38         | 68.75         | 65.63         | <b>73.438</b> | 64.06         | 68.75         | 70.31         | <b>73.438</b> | 0.58        | 0.71        | 0.66 | <b>0.77</b> | 0.66        | 0.71        | 0.72        | 0.74        |
| DSAM             | 67.11         | 71.05         | 72.37         | 71.05         | 72.37         | 72.37         | <b>73.684</b> | 72.37         | 0.66        | 0.72        | 0.71 | <b>0.81</b> | 0.73        | 0.75        | 0.74        | 0.78        |
| Info gain (FR3)  |               |               |               |               |               |               |               |               |             |             |      |             |             |             |             |             |
| OD               | 71.67         | 88.74         | 87.71         | 90.10         | 88.40         | 89.08         | 88.74         | <b>90.785</b> | 0.76        | <b>0.81</b> | 0.7  | 0.79        | 0.74        | 0.78        | 0.72        | 0.75        |
| SMOTE            | 58.24         | 76.63         | 86.40         | <b>91.188</b> | 77.40         | 76.63         | 83.14         | 87.93         | 0.78        | 0.84        | 0.86 | <b>0.96</b> | 0.83        | 0.84        | 0.9         | 0.93        |
| BSMOTE           | 72.41         | 81.23         | 91.00         | 90.42         | 80.84         | 79.12         | 83.33         | <b>91.954</b> | 0.85        | 0.88        | 0.91 | <b>0.97</b> | 0.89        | 0.88        | 0.92        | 0.95        |
| SSMOTE           | 64.56         | 83.91         | 85.44         | <b>89.464</b> | 84.67         | 84.10         | 87.17         | 86.78         | 0.87        | 0.9         | 0.85 | <b>0.95</b> | 0.89        | 0.9         | 0.92        | 0.93        |
| SMOTEE           | 70.60         | 74.18         | 88.74         | <b>90.659</b> | 75.82         | 64.56         | 85.99         | 87.09         | 0.86        | 0.75        | 0.88 | <b>0.95</b> | 0.86        | 0.73        | 0.94        | 0.92        |
| SMOTETO          | 61.22         | 79.92         | 89.17         | <b>93.11</b>  | 80.91         | 77.56         | 86.81         | 91.93         | 0.81        | 0.85        | 0.89 | <b>0.97</b> | 0.88        | 0.85        | 0.94        | 0.96        |
| USAM             | 88.12         | 90.61         | 92.15         | <b>94.253</b> | 92.34         | 90.42         | 91.76         | 93.30         | 0.95        | 0.93        | 0.92 | <b>0.97</b> | 0.95        | 0.93        | <b>0.97</b> | 0.94        |
| RSAM             | 64.06         | <b>76.563</b> | 64.06         | 65.63         | 64.06         | 73.44         | 56.25         | 65.63         | 0.67        | <b>0.81</b> | 0.64 | 0.72        | 0.7         | <b>0.81</b> | 0.61        | 0.71        |
| DSAM             | 61.84         | 71.05         | 72.37         | 72.37         | <b>75</b>     | 72.37         | 59.21         | 69.74         | 0.64        | 0.76        | 0.71 | 0.74        | 0.76        | <b>0.81</b> | 0.53        | 0.7         |
| OneR (FR4)       |               |               |               |               |               |               |               |               |             |             |      |             |             |             |             |             |
| OD               | 65.53         | 88.74         | 87.37         | <b>89.761</b> | 89.08         | 89.08         | 89.08         | 88.06         | 0.71        | <b>0.81</b> | 0.68 | 0.78        | 0.74        | 0.76        | 0.63        | 0.73        |
| SMOTE            | 50.38         | 58.62         | 80.65         | 82.76         | 66.28         | 59.39         | 59.58         | <b>85.632</b> | 0.69        | 0.68        | 0.81 | <b>0.91</b> | 0.77        | 0.66        | 0.63        | <b>0.91</b> |
| BSMOTE           | 52.49         | 61.88         | 83.91         | 83.53         | 69.73         | 60.92         | 54.22         | <b>86.015</b> | 0.66        | 0.67        | 0.84 | 0.91        | 0.78        | 0.68        | 0.58        | <b>0.93</b> |
| SSMOTE           | 72.41         | 84.29         | 87.17         | 88.89         | 83.91         | 84.29         | 75.86         | <b>90.613</b> | 0.87        | 0.91        | 0.87 | <b>0.95</b> | 0.91        | 0.91        | 0.83        | <b>0.95</b> |
| SMOTEE           | 81.59         | 85.17         | 91.76         | 90.39         | 85.44         | 84.89         | 62.64         | <b>92.857</b> | 0.89        | 0.93        | 0.91 | <b>0.96</b> | 0.93        | 0.93        | 0.8         | <b>0.96</b> |
| SMOTETO          | 69.29         | 79.33         | 87.60         | <b>91.142</b> | 77.95         | 78.15         | 84.25         | 90.16         | 0.83        | 0.85        | 0.88 | <b>0.97</b> | 0.87        | 0.85        | 0.93        | 0.95        |
| USAM             | 88.12         | 91.00         | 91.19         | <b>94.636</b> | 92.53         | 90.42         | 91.76         | 92.34         | 0.96        | 0.93        | 0.91 | <b>0.97</b> | 0.95        | 0.93        | <b>0.97</b> | 0.95        |
| RSAM             | 65.63         | 73.44         | <b>75</b>     | 65.63         | 64.06         | <b>75</b>     | 68.75         | 70.31         | 0.73        | <b>0.79</b> | 0.75 | 0.7         | 0.72        | <b>0.79</b> | 0.73        | 0.74        |
| DSAM             | 65.79         | 68.42         | 67.11         | 68.42         | <b>73.684</b> | 71.05         | 65.79         | 65.79         | 0.64        | <b>0.77</b> | 0.66 | 0.68        | 0.76        | 0.75        | 0.64        | 0.73        |
| AdaBoost (FS2)   |               |               |               |               |               |               |               |               |             |             |      |             |             |             |             |             |
| OD               | 88.40         | 89.08         | 84.98         | 89.08         | <b>90.785</b> | 89.08         | 89.42         | 88.74         | 0.75        | 0.78        | 0.64 | 0.7         | 0.72        | <b>0.82</b> | 0.78        | 0.69        |
| SMOTE            | 72.80         | 78.93         | 84.48         | <b>85.824</b> | 78.16         | 78.16         | 53.45         | 82.38         | 0.78        | 0.8         | 0.84 | <b>0.9</b>  | 0.8         | 0.8         | 0.54        | 0.87        |
| BSMOTE           | 79.12         | 80.65         | 86.78         | <b>90.038</b> | 82.57         | 82.18         | 84.29         | <b>90.038</b> | 0.85        | 0.86        | 0.87 | <b>0.95</b> | 0.88        | 0.86        | 0.92        | 0.93        |
| SSMOTE           | 84.67         | 87.55         | 89.27         | <b>92.337</b> | 88.70         | 87.74         | 86.02         | 90.61         | 0.9         | 0.9         | 0.89 | <b>0.97</b> | 0.93        | 0.91        | 0.94        | 0.94        |
| SMOTEE           | 84.89         | 83.79         | 92.86         | <b>93.407</b> | 87.09         | 78.30         | 86.81         | 93.13         | 0.94        | 0.9         | 0.92 | <b>0.98</b> | 0.92        | 0.88        | 0.95        | 0.96        |
| SMOTETO          | 73.43         | 77.95         | 82.09         | <b>86.024</b> | 77.76         | 78.74         | 80.32         | 81.10         | 0.78        | 0.8         | 0.83 | <b>0.91</b> | 0.81        | 0.8         | 0.88        | 0.84        |
| USAM             | 75.48         | 68.58         | 93.49         | 93.10         | 83.91         | 75.10         | 93.30         | <b>93.87</b>  | 0.86        | 0.65        | 0.93 | <b>0.94</b> | 0.77        | 0.65        | <b>0.94</b> | <b>0.94</b> |
| RSAM             | <b>75</b>     | <b>75</b>     | 62.50         | 67.19         | 67.19         | 67.19         | 68.75         | 70.31         | <b>0.84</b> | 0.83        | 0.63 | 0.77        | 0.75        | 0.82        | 0.77        | 0.75        |
| DSAM             | <b>71.053</b> | 57.90         | 59.21         | 61.84         | 67.11         | 61.84         | 56.58         | 56.58         | <b>0.7</b>  | 0.44        | 0.58 | 0.6         | 0.59        | 0.36        | 0.42        | 0.66        |
| CFS (FS3)        |               |               |               |               |               |               |               |               |             |             |      |             |             |             |             |             |
| OD               | 88.06         | <b>89.079</b> | 84.30         | 83.96         | <b>89.079</b> | <b>89.079</b> | <b>89.079</b> | 85.32         | <b>0.74</b> | 0.7         | 0.52 | 0.69        | 0.49        | 0.51        | 0.69        | 0.64        |
| SMOTE            | 74.33         | 77.20         | 88.51         | <b>90.613</b> | 80.27         | 78.16         | 83.14         | 87.17         | 0.84        | 0.86        | 0.89 | <b>0.96</b> | 0.89        | 0.85        | 0.91        | 0.94        |
| BSMOTE           | 79.12         | 80.08         | 86.59         | <b>91.188</b> | 81.80         | 78.35         | 83.91         | 88.12         | 0.88        | 0.87        | 0.87 | <b>0.97</b> | 0.92        | 0.87        | 0.93        | 0.94        |
| SSMOTE           | 80.65         | 84.87         | 86.59         | <b>93.295</b> | 83.91         | 84.29         | 85.06         | 90.04         | 0.9         | 0.9         | 0.87 | <b>0.97</b> | 0.92        | 0.9         | 0.94        | 0.94        |
| SMOTEE           | 84.34         | 85.71         | <b>94.231</b> | 93.41         | 85.44         | 86.54         | 90.39         | <b>94.231</b> | 0.92        | 0.94        | 0.94 | <b>0.98</b> | 0.93        | 0.93        | 0.97        | <b>0.98</b> |

(continued on next page)

Table 9 (continued).

|         | Accuracy |             |             |               |       |       |       |               | AUC  |      |             |             |      |      |             |      |
|---------|----------|-------------|-------------|---------------|-------|-------|-------|---------------|------|------|-------------|-------------|------|------|-------------|------|
|         | GNB      | LOGR        | DT          | RF            | SVCR  | SVCL  | BAG   | ADAB          | GNB  | LOGR | DT          | RF          | SVCR | SVCL | BAG         | ADAB |
| SMOTETO | 74.02    | 77.56       | 86.02       | <b>90.748</b> | 80.12 | 76.77 | 80.91 | <b>90.748</b> | 0.85 | 0.85 | 0.86        | <b>0.96</b> | 0.89 | 0.85 | 0.92        | 0.95 |
| USAM    | 91.19    | 89.85       | 92.53       | 92.53         | 92.72 | 89.46 | 91.57 | <b>93.487</b> | 0.95 | 0.93 | 0.93        | <b>0.97</b> | 0.95 | 0.92 | <b>0.97</b> | 0.96 |
| RSAM    | 53.13    | <b>62.5</b> | <b>62.5</b> | 57.81         | 60.94 | 48.44 | 51.56 | 56.25         | 0.54 | 0.6  | <b>0.63</b> | 0.62        | 0.55 | 0.36 | 0.56        | 0.5  |
| DSAM    | 68.42    | 63.16       | 72.37       | <b>73.684</b> | 61.84 | 57.90 | 67.11 | 72.37         | 0.68 | 0.5  | 0.72        | <b>0.78</b> | 0.6  | 0.42 | 0.64        | 0.72 |

4.3. What effect do the use of different combinations of feature selection and sampling approaches have on the performance of the developed SFP model?

#### 4.3.1. Performance value

In this research, 8 different sampling strategies are employed to balance the 54 open-source projects with faulty software metrics, which resulted in 432 balanced datasets. To deal with the feature redundancy problem, 10 feature selection techniques have been induced. For comparative analysis and validation of the performance of produced SFP models with the most effective combinations, we have considered the dataset with all feature metrics and unsampled datasets, this resulted in the generation of 5346 variants of input datasets. The performance value of developed SFP models with/without sampling and feature selection techniques is represented in Table 9, considering the space constraint we have illustrated the outcome of randomly selected project 1 (Ant 1.5). Table 9 shows the AUC value of the model trained using the RF classifier without the application of the sampling technique and feature selection technique is 0.77. Whereas with an application of SMOTEE and AdaBoost feature subset selection method improved AUC value is 0.98. Moreover, the secured AUC value of the RF classifier for the same project after the application of the SSOTTE and FR4 is 0.95. This shows how changing the combinations of sampling and feature selection strategies can affect the performance and predictability of trained SFP models. Additionally, even though the FS3 feature selection techniques individually outperformed the competition, however, SMOTEE and FS5 with fewer extracted features secured the highest AUC value against the Project 1 dataset modeled using the RF classifier. This makes it clear that our extensive experimentation enabled us to choose the ideal combination for improving the developed SFP model's accuracy and performance.

The outcome observations for the represented project show the improved difference in AUC value from 0.67 (without the implication of sampling and feature selection method) to 0.94 i.e. 40% improvement of DT classifier achieved post implication of SMOTEE and FS3. Fig. 6; manifests the AUC diagram of Project Ant 1.5, AUC close to one indicates a high level of separability and it results in an excellent model. A poor model has an AUC near zero, indicating the worst measure of separability. Fig. 6 shows that the models with the curve closest to the uppermost left corner perform better. And from Fig. 6, we observed the models trained using mentioned classifiers post-application of SMOTEE sampling approach outperformed.

The above results based on AUC values show the improved difference in AUC value achieved post implication of feature selection and sampling approaches.

### 5. Comparative analysis of combined techniques (Feature selection and sampling)

For the proposed model the balanced class-level metrics datasets were generated by applying 8 sampling techniques individually. On each of these generated datasets, we have applied 10 feature selection techniques and one relevant feature with all feature metrics and computed the performance metrics (accuracy & AUC) under each classifier for every project. The significance and reliability of the built models are compared using statistical and AUC analyses. The intensive assessment and performance of these techniques individually are presented in this segment.

5.1. Is the outcome of feature-ranking and feature-subset selection strategies statistically equivalent?

In this study, we employed five feature ranking strategies and five feature subset selection techniques against 54 open-source datasets to draw a generalized conclusion on the best-performing feature selection methods. We then used the reduced class-level metrics as inputs to the models we built to forecast the incorrect class. Statistical and AUC studies were used to determine the significance and reliability of various feature selection strategies.

**Comparison of feature selection techniques using descriptive statistics and boxplots:** Fig. 7 depicts the boxplot for the accuracy and AUC of 10 feature selection techniques applied on balanced data and data without any feature selection techniques applied (Refer to "AM" in Fig. 7). The descriptive statistics of all employed feature selection techniques are presented in Table 10. The following conclusions can be drawn from Fig. 7 and Table 10:

- All the models give reasonable accuracies ranging between 70%–80% and AUC values between 0.75–0.85.
- The median accuracy (%) and median AUC value for FR5 are higher than other techniques.
- The model trained after applying FR5 performs better than the rest of the techniques.

**Comparison of the feature selection technique using statistical tests:** To compute the performance analysis, we used two statistical tests: The Friedman Test and the Wilcoxon Signed Rank Test. The Wilcoxon Signed Rank Test determines whether a statistically significant difference exists between various feature selection strategies used to reduce the dimensionality of balanced source code metrics. Friedman test algorithms allocate mean ranks based on their AUC performance score. The lowest the mean rank the greater the performance of the technique.

Table 11 demonstrates the hypothesis testing between 10 feature selection techniques and with all feature metric pairs. For Wilcoxon Sign Rank Test considering the null hypothesis (H0) shows that there is no significant difference in the performance of alternative feature selection methods. Reject H0 only if  $p \leq 0.05$ . Table 11 shows that most pairs can reject the null hypothesis. Therefore, the feature selection techniques used to reduce the dimensions of balanced source code metrics appear to be very different. As a result, all ten feature selection techniques should be considered when developing the best-performing predictive model for identifying defective classes.

The Friedman test results for feature selection approaches are shown in Table 11, which aids us in determining the best feature selection approach. From Table 11, we observe the lowest mean rank is 4.96 for the relevant features where all the feature metrics (AM) were considered. At the same time, we found the mean rank of FS3 with reduced dimensionality of the balanced source code dataset is 5.37 which is closer to the mean rank of AM. And the mean rank for other feature selection techniques is also in the fair range. We deduced from all of this that different feature selection techniques impact the performance of fault prediction models. As a result, to build the SFP models, all feature selection techniques must be used.

The feature selection techniques used to reduce the dimensions for fault prediction appear to be significantly different. The model trained after applying feature selection techniques has a better ability for fault prediction.

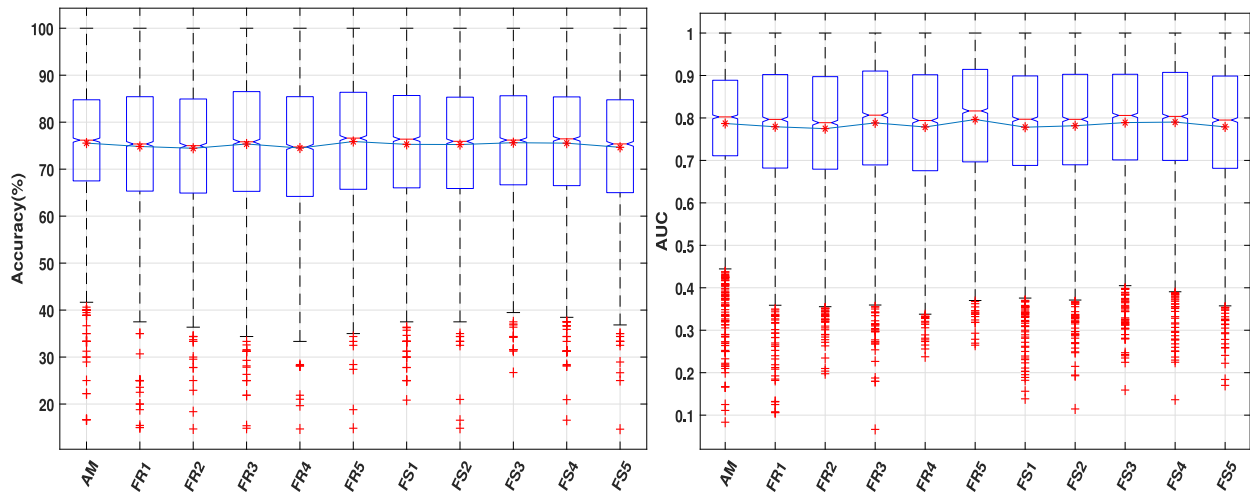


Fig. 7. Accuracy and AUC boxplots of feature selection techniques.

Table 10

Employed feature selection techniques' descriptive statistics.

|     | Accuracy |        |       |        |       |       | AUC  |        |      |      |      |      |
|-----|----------|--------|-------|--------|-------|-------|------|--------|------|------|------|------|
|     | Min      | Median | Mean  | Max    | Q3    | Q1    | Min  | Median | Mean | Max  | Q3   | Q1   |
| AM  | 16.67    | 76.18  | 75.55 | 100.00 | 84.77 | 67.50 | 0.08 | 0.80   | 0.79 | 1.00 | 0.89 | 0.71 |
| FR1 | 15.00    | 75.36  | 74.83 | 100.00 | 85.42 | 65.33 | 0.11 | 0.80   | 0.78 | 1.00 | 0.90 | 0.68 |
| FR2 | 14.70    | 75.00  | 74.45 | 100.00 | 84.94 | 64.91 | 0.20 | 0.79   | 0.77 | 1.00 | 0.90 | 0.68 |
| FR3 | 14.85    | 75.82  | 75.36 | 100.00 | 86.51 | 65.28 | 0.07 | 0.81   | 0.79 | 1.00 | 0.91 | 0.69 |
| FR4 | 14.70    | 74.73  | 74.50 | 100.00 | 85.43 | 64.20 | 0.24 | 0.79   | 0.78 | 1.00 | 0.90 | 0.68 |
| FR5 | 14.85    | 76.61  | 75.88 | 100.00 | 86.36 | 65.71 | 0.26 | 0.82   | 0.80 | 1.00 | 0.91 | 0.70 |
| FS1 | 20.83    | 76.39  | 75.28 | 100.00 | 85.68 | 66.02 | 0.14 | 0.80   | 0.78 | 1.00 | 0.90 | 0.69 |
| FS2 | 14.85    | 75.93  | 75.24 | 100.00 | 85.32 | 65.88 | 0.11 | 0.80   | 0.78 | 1.00 | 0.90 | 0.69 |
| FS3 | 26.70    | 76.19  | 75.63 | 100.00 | 85.62 | 66.67 | 0.16 | 0.81   | 0.79 | 1.00 | 0.90 | 0.70 |
| FS4 | 16.56    | 76.45  | 75.53 | 100.00 | 85.38 | 66.49 | 0.14 | 0.80   | 0.79 | 1.00 | 0.91 | 0.70 |
| FS5 | 14.65    | 75.38  | 74.65 | 100.00 | 84.77 | 65.00 | 0.17 | 0.80   | 0.78 | 1.00 | 0.90 | 0.68 |

Table 11

AUC: Statistical and Friedman test results of feature selection techniques.

|     | AM   | FR1  | FR2  | FR3  | FR4  | FR5  | FS1  | FS1  | FS3  | FS4  | FS5  | Mean-Rank |
|-----|------|------|------|------|------|------|------|------|------|------|------|-----------|
| AM  | 1.00 | 0.07 | 0.00 | 0.29 | 0.01 | 0.00 | 0.15 | 0.29 | 0.21 | 0.15 | 0.04 | 4.96      |
| FR1 | 0.07 | 1.00 | 0.14 | 0.01 | 0.53 | 0.00 | 0.81 | 0.48 | 0.00 | 0.00 | 0.71 | 6.27      |
| FR2 | 0.00 | 0.14 | 1.00 | 0.00 | 0.40 | 0.00 | 0.08 | 0.03 | 0.00 | 0.00 | 0.26 | 6.42      |
| FR3 | 0.29 | 0.01 | 0.00 | 1.00 | 0.00 | 0.05 | 0.01 | 0.04 | 1.00 | 0.73 | 0.00 | 6.28      |
| FR4 | 0.01 | 0.53 | 0.40 | 0.00 | 1.00 | 0.00 | 0.42 | 0.19 | 0.00 | 0.00 | 0.81 | 6.62      |
| FR5 | 0.00 | 0.00 | 0.00 | 0.05 | 0.00 | 1.00 | 0.00 | 0.00 | 0.05 | 0.11 | 0.00 | 6.03      |
| FS1 | 0.15 | 0.81 | 0.08 | 0.01 | 0.42 | 0.00 | 1.00 | 0.66 | 0.01 | 0.00 | 0.60 | 5.91      |
| FS2 | 0.29 | 0.48 | 0.03 | 0.04 | 0.19 | 0.00 | 0.66 | 1.00 | 0.03 | 0.01 | 0.31 | 5.78      |
| FS3 | 0.21 | 0.00 | 0.00 | 1.00 | 0.00 | 0.05 | 0.01 | 0.03 | 1.00 | 0.77 | 0.00 | 5.37      |
| FS4 | 0.15 | 0.00 | 0.00 | 0.73 | 0.00 | 0.11 | 0.00 | 0.01 | 0.77 | 1.00 | 0.00 | 5.70      |
| FS5 | 0.04 | 0.71 | 0.26 | 0.00 | 0.81 | 0.00 | 0.60 | 0.31 | 0.00 | 0.00 | 1.00 | 6.66      |

## 5.2. Which sampling method gives efficient and effective performance?

We used eight sampling techniques to tackle the class imbalance problem, and the resulting balanced datasets are fed into the built models to predict the faulty class. The significance and reliability of these employed sampling strategies were determined using statistical and AUC analyses to make a broad generalization regarding the most effective sampling techniques.

**Comparison of sampling techniques using descriptive statistics and boxplots:** Fig. 8 shows the boxplot for the accuracy and AUC of eight sampling techniques applied on 54 open-source projects datasets along with a box plot for original data without any sampling techniques applied (refer to "OD" in Fig. 8). Table 12 elucidates the descriptive statistics of accuracy (%) and AUC values for sampling techniques. From Fig. 8 and Table 12, we deduce the following observations:

- The median accuracy (%) and median AUC value for the SMOTEE method are higher than other techniques.

- The model trained after applying SMOTEE performs better than the rest of the techniques.

## What is the impact of sampling techniques on the predictability of SFP models?

**Comparison of the sampling technique using statistical tests:** To compute statistically significant differences between the various pairings of sampling techniques' performance, we employed the Wilcoxon signed-rank test in this study. The Wilcoxon Sign Rank test's null hypothesis (H0) is that "there is no significant difference in the performance of different sampling strategies". If  $p \leq 0.05$ , we reject the null hypothesis. Table 11 shows that the null hypothesis can be rejected for all pairs of sampling methods, except for (SMOTE, BSMOTE), (BSMOTE, SSMOTE), and (DSAM, RSAM), which all have  $p$  values  $>0.05$ . As a result, we conclude that the performance of the majority of sampling approaches appears to differ significantly.

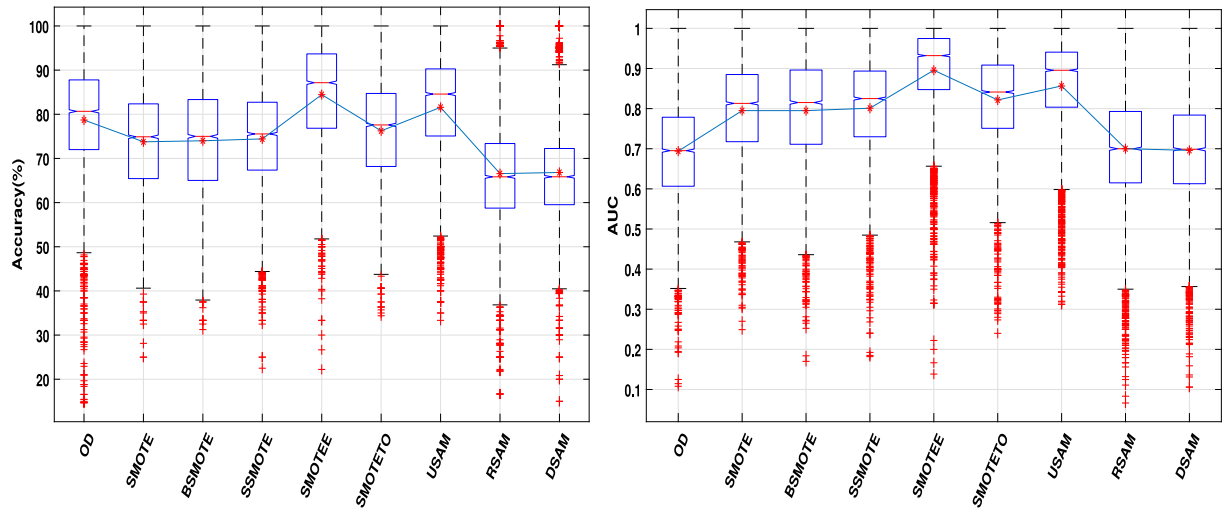


Fig. 8. Accuracy and AUC boxplots of sampling techniques.

Table 12

All sampling techniques' descriptive statistics (OD: Original Datasets).

|         | Accuracy |        |       |        |       |       | AUC  |        |      |      |      |      |
|---------|----------|--------|-------|--------|-------|-------|------|--------|------|------|------|------|
|         | Min      | Median | Mean  | Max    | Q3    | Q1    | Min  | Median | Mean | Max  | Q3   | Q1   |
| OD      | 14.65    | 80.67  | 78.70 | 100.00 | 87.78 | 72.00 | 0.11 | 0.70   | 0.69 | 1.00 | 0.78 | 0.61 |
| SMOTE   | 25.00    | 74.90  | 73.77 | 100.00 | 82.36 | 65.43 | 0.25 | 0.81   | 0.80 | 1.00 | 0.88 | 0.72 |
| BSMOTE  | 31.25    | 75.00  | 74.00 | 100.00 | 83.33 | 65.02 | 0.17 | 0.82   | 0.79 | 1.00 | 0.90 | 0.71 |
| SSMOTE  | 22.50    | 75.55  | 74.42 | 100.00 | 82.73 | 67.36 | 0.18 | 0.83   | 0.80 | 1.00 | 0.89 | 0.73 |
| SMOTEE  | 22.22    | 87.15  | 84.45 | 100.00 | 93.67 | 76.84 | 0.14 | 0.93   | 0.90 | 1.00 | 0.97 | 0.85 |
| SMOTETO | 34.38    | 77.59  | 76.25 | 100.00 | 84.72 | 68.18 | 0.24 | 0.84   | 0.82 | 1.00 | 0.91 | 0.75 |
| USAM    | 33.33    | 84.58  | 81.57 | 100.00 | 90.28 | 75.08 | 0.31 | 0.90   | 0.86 | 1.00 | 0.94 | 0.80 |
| RSAM    | 16.67    | 65.83  | 66.57 | 100.00 | 73.38 | 58.75 | 0.07 | 0.70   | 0.70 | 1.00 | 0.79 | 0.61 |
| DSAM    | 15.00    | 65.84  | 66.82 | 100.00 | 72.25 | 59.52 | 0.11 | 0.70   | 0.70 | 1.00 | 0.78 | 0.61 |

Table 13

AUC: Statistical and Friedman test results of sampling techniques.

|         | OD   | SMOTE | BSMOTE | SSMOTE | SMOTEE | SMOTETO | USAM | RSAM | DSAM | Mean-Rank |
|---------|------|-------|--------|--------|--------|---------|------|------|------|-----------|
| OD      | 1.00 | 0.00  | 0.00   | 0.00   | 0.00   | 0.00    | 0.00 | 0.00 | 0.16 | 7.16      |
| SMOTE   | 0.00 | 1.00  | 0.21   | 0.00   | 0.00   | 0.00    | 0.00 | 0.00 | 0.00 | 5.22      |
| BSMOTE  | 0.00 | 0.21  | 1.00   | 0.03   | 0.00   | 0.00    | 0.00 | 0.00 | 0.00 | 5.06      |
| SSMOTE  | 0.00 | 0.00  | 0.03   | 1.00   | 0.00   | 0.00    | 0.00 | 0.00 | 0.00 | 4.83      |
| SMOTEE  | 0.00 | 0.00  | 0.00   | 0.00   | 1.00   | 0.00    | 0.00 | 0.00 | 0.00 | 1.87      |
| SMOTETO | 0.00 | 0.00  | 0.00   | 0.00   | 0.00   | 1.00    | 0.00 | 0.00 | 0.00 | 4.06      |
| USAM    | 0.00 | 0.00  | 0.00   | 0.00   | 0.00   | 0.00    | 1.00 | 0.00 | 0.00 | 2.87      |
| RSAM    | 0.00 | 0.00  | 0.00   | 0.00   | 0.00   | 0.00    | 0.00 | 1.00 | 0.10 | 6.89      |
| DSAM    | 0.16 | 0.00  | 0.00   | 0.00   | 0.00   | 0.00    | 0.00 | 0.10 | 1.00 | 7.04      |

Table 13 indicates the Friedman test findings for sampling approaches. The Friedman test assists us in finding the most appropriate sampling strategy. Table 13 parade that the SMOTEE approach has the lowest mean rank of 1.87. We deduce that SMOTEE has the best overall performance in handling the class imbalance problem.

The model trained after applying SMOTEE performs better than the rest of the techniques. Friedman test demonstrates that the fault prediction models trained on sampled data using sampling approaches have significant improvement in their performance.

### 5.3. Comparative analysis: Different classifiers

In this study, eight different classifiers are employed to train models for detecting faulty source code metrics. Using box plots, descriptive

statistics, and statistical test analyses, we assessed the implications and dependabilities of these classifiers on a variety of datasets.

**Comparison of classifier models using descriptive statistics and boxplots:** The boxplot of the eight distinct classifiers used to train the SFP models with the accuracy and AUC performance parameters is shown in Fig. 9. And Table 14 presents descriptive statistics for the eight classifiers mentioned before. The following conclusions can be drawn from Fig. 9 and Table 14:

- The median accuracy (%) and median AUC value for the RF (Random Forest) classifier are higher than other techniques.
- From the descriptive statistics, we observed that the Random Forest classifier performs better than the rest of the other classifiers.

**Comparison of the classification techniques using statistical tests:** As stated in the preceding sections, the Wilcoxon Signed Rank Sum Test and Friedman Test were employed to analyze the statistical variations among the pairs of different classifiers. For the Wilcoxon Sign

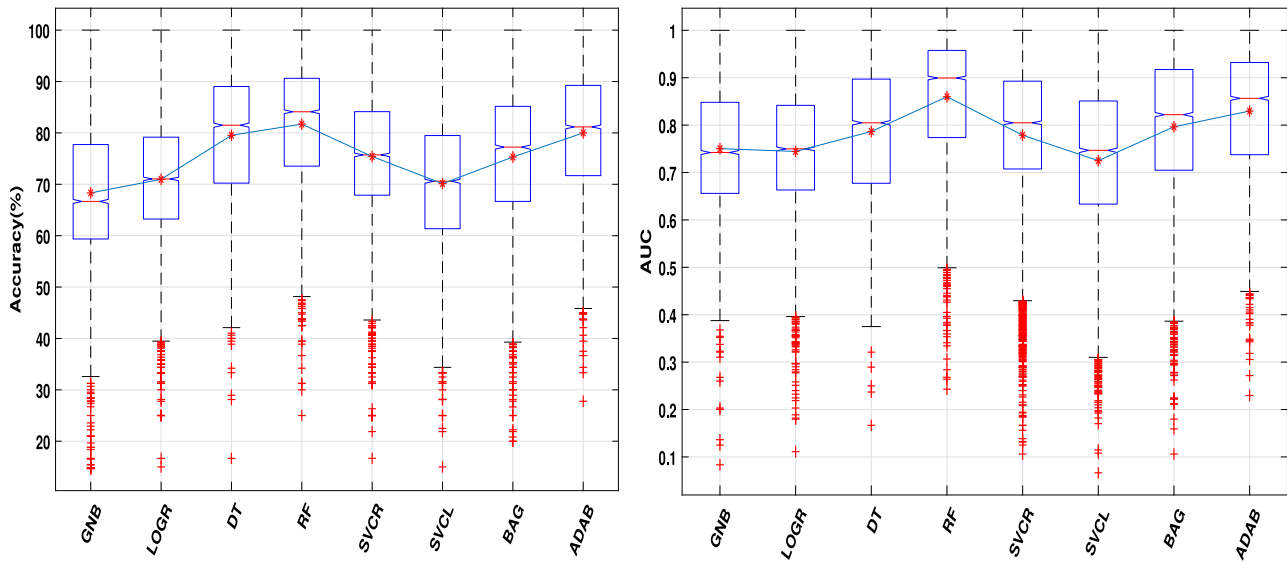


Fig. 9. Accuracy and AUC boxplots of employed eight classifiers.

**Table 14**  
Descriptive statistics of employed eight classifiers.

|      | Accuracy |        |       |        |       |       | AUC  |        |      |      |      |      |
|------|----------|--------|-------|--------|-------|-------|------|--------|------|------|------|------|
|      | Min      | Median | Mean  | Max    | Q3    | Q1    | Min  | Median | Mean | Max  | Q3   | Q1   |
| GNB  | 14.65    | 66.67  | 68.32 | 100.00 | 77.73 | 59.35 | 0.08 | 0.74   | 0.75 | 1.00 | 0.85 | 0.66 |
| LOGR | 15.00    | 71.04  | 70.97 | 100.00 | 79.17 | 63.24 | 0.11 | 0.75   | 0.74 | 1.00 | 0.84 | 0.66 |
| DT   | 16.67    | 81.50  | 79.53 | 100.00 | 89.02 | 70.22 | 0.17 | 0.80   | 0.79 | 1.00 | 0.90 | 0.68 |
| RF   | 25.00    | 84.11  | 81.72 | 100.00 | 90.62 | 73.53 | 0.24 | 0.90   | 0.86 | 1.00 | 0.96 | 0.77 |
| SVCR | 16.67    | 75.71  | 75.40 | 100.00 | 84.13 | 67.88 | 0.11 | 0.80   | 0.78 | 1.00 | 0.89 | 0.71 |
| SVCL | 15.00    | 70.59  | 70.13 | 100.00 | 79.49 | 61.36 | 0.07 | 0.75   | 0.72 | 1.00 | 0.85 | 0.63 |
| BAG  | 20.00    | 77.24  | 75.31 | 100.00 | 85.16 | 66.67 | 0.11 | 0.82   | 0.80 | 1.00 | 0.92 | 0.70 |
| ADAB | 27.78    | 81.17  | 80.00 | 100.00 | 89.24 | 71.68 | 0.23 | 0.86   | 0.83 | 1.00 | 0.93 | 0.74 |

**Table 15**  
AUC: Statistical and Friedman test results of all classifiers.

|      | GNB  | LOGR | DT   | RF   | SVCR | SVCL | BAG  | ADAB | Mean-Rank |
|------|------|------|------|------|------|------|------|------|-----------|
| GNB  | 1.00 | 0.80 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 5.82      |
| LOGR | 0.80 | 1.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 5.70      |
| DT   | 0.00 | 0.00 | 1.00 | 0.00 | 0.90 | 0.00 | 0.00 | 0.00 | 5.15      |
| RF   | 0.00 | 0.00 | 0.00 | 1.00 | 0.00 | 0.00 | 0.00 | 0.00 | 1.93      |
| SVCR | 0.00 | 0.00 | 0.90 | 0.00 | 1.00 | 0.00 | 0.00 | 0.00 | 4.40      |
| SVCL | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 1.00 | 0.00 | 0.00 | 5.84      |
| BAG  | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 1.00 | 0.00 | 3.93      |
| ADAB | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 1.00 | 3.23      |

Rank, the null hypothesis (H0) states that there is no great difference in overall performance, trained software fault prediction models using various classifiers. If the value of  $p \leq 0.05$ , we reject the null hypothesis. From Table 15, we derived the null hypothesis that can be rejected for all pairs of classifiers except for (SVCR & DT) and (LOGR & GNB), which have a  $p$ -value  $> 0.05$ . As a result, we concluded the performance of various classifier algorithms differs statistically. The Friedman test results for classification approaches are shown in Table 15. This test aids us in determining the best classification approach. From Table 15, we observe the lowest mean rank is 1.93 for the Random Forest (RF) classifier. In this study, we have determined the RF classifier has shown significant performance in compared to other classifiers.

**The Random Forest classifier performs better than the rest of the other classifiers. Hypothesis test demonstrates that the performance of various classifier algorithms differs statistically.**

#### 5.4. Is the outcome of various feature selection methods combined with different sampling techniques statistically equivalent?

In this research work, we analyzed and evaluated the efficacy of combinations of 11 feature selections (1 for All metrics) and 9 data sampling approaches (1 unsampled dataset) on the performance of SFP models. Utilizing statistical and AUC analysis, the validity and reliability of these used combinations were determined.

**Comparison of different sampling techniques with feature selection using descriptive statistics and boxplots:** Figs. 10(a) and 10(b) display the boxplot for accuracy and AUC of nine sampling techniques with all metrics and the application of feature selection with the accuracy and AUC performance parameters. To compute descriptive measures, first, we determined it for 8 sampling techniques and one unsampled dataset with original data i.e. with all metrics and then we calculated it for employed 8 sampling techniques and one unsampled dataset with feature selection approaches. Table 16 presents descriptive statistics of 9 sampling approaches with/without feature selection. The following inferences can be made from Table 16 and Fig. 10(a):

- The median accuracy (%) value for the SMOTEE with all metrics (AM) is higher than other techniques
- SMOTEE with feature selection secured the highest AUC value of 0.94, which indicates the improved efficiency and predictability of the SFP model post the application of SMOTEE with feature selection
- Figs. 10(a) and 10(b) depicts that, with reduced metrics size, we were still able to achieve the same accuracy and AUC for SMOTEE with AM and SMOTEE with FS

**Comparison of different sampling techniques with feature selection using statistical tests:** In this study, we used the Wilcoxon

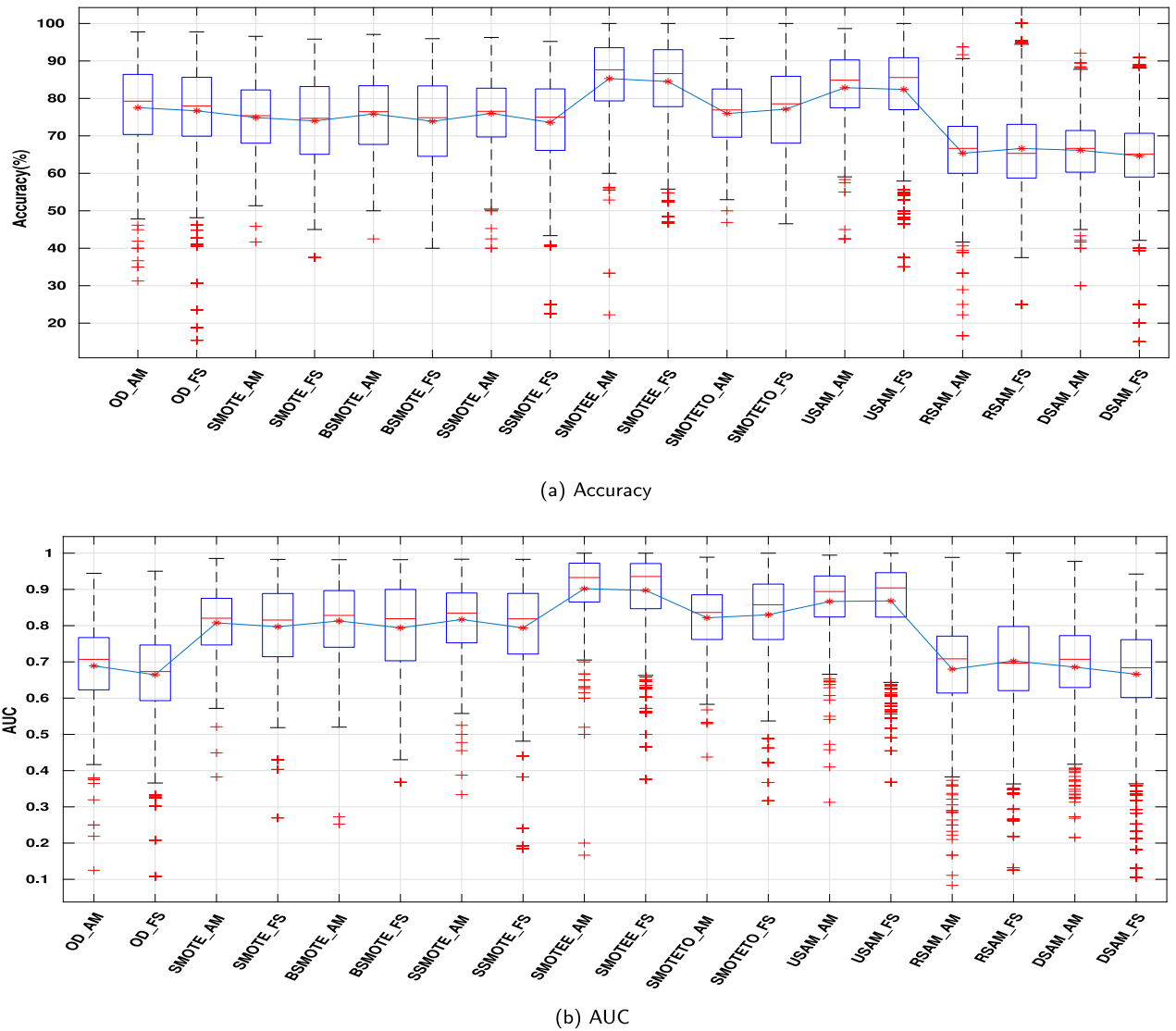


Fig. 10. Accuracy and AUC boxplots of data sampling with feature selection techniques.

Table 16

Descriptive statistics of data sampling with feature selection techniques.

|            | Accuracy |        |       |        |       |       | AUC  |        |      |      |      |      |
|------------|----------|--------|-------|--------|-------|-------|------|--------|------|------|------|------|
|            | Min      | Median | Mean  | Max    | Q3    | Q1    | Min  | Median | Mean | Max  | Q3   | Q1   |
| OD_AM      | 31.25    | 79.26  | 77.53 | 97.76  | 86.41 | 70.37 | 0.13 | 0.71   | 0.69 | 0.94 | 0.77 | 0.62 |
| OD_FS      | 15.45    | 78.01  | 76.69 | 97.76  | 85.63 | 69.91 | 0.11 | 0.67   | 0.66 | 0.95 | 0.75 | 0.59 |
| SMOTE_AM   | 41.67    | 75.38  | 74.90 | 96.57  | 82.26 | 68.05 | 0.38 | 0.82   | 0.81 | 0.99 | 0.88 | 0.75 |
| SMOTE_FS   | 37.50    | 74.69  | 73.99 | 95.83  | 83.16 | 65.08 | 0.27 | 0.82   | 0.80 | 0.98 | 0.89 | 0.71 |
| BSMOTE_AM  | 42.50    | 76.46  | 75.86 | 97.09  | 83.38 | 67.72 | 0.25 | 0.83   | 0.81 | 0.98 | 0.90 | 0.74 |
| BSMOTE_FS  | 40.00    | 74.84  | 73.89 | 95.95  | 83.33 | 64.54 | 0.37 | 0.82   | 0.79 | 0.98 | 0.90 | 0.70 |
| SSMOTE_AM  | 40.00    | 76.56  | 76.01 | 96.27  | 82.73 | 69.71 | 0.33 | 0.83   | 0.82 | 0.98 | 0.89 | 0.75 |
| SSMOTE_FS  | 22.50    | 75.00  | 73.59 | 95.21  | 82.54 | 66.13 | 0.19 | 0.82   | 0.79 | 0.98 | 0.89 | 0.72 |
| SMOTEE_AM  | 22.22    | 87.64  | 85.29 | 100.00 | 93.54 | 79.33 | 0.17 | 0.93   | 0.90 | 1.00 | 0.97 | 0.87 |
| SMOTEE_FS  | 46.78    | 86.62  | 84.50 | 100.00 | 93.00 | 77.80 | 0.38 | 0.94   | 0.90 | 1.00 | 0.97 | 0.85 |
| SMOTETO_AM | 46.88    | 76.92  | 75.96 | 96.00  | 82.51 | 69.65 | 0.44 | 0.84   | 0.82 | 0.99 | 0.89 | 0.76 |
| SMOTETO_FS | 46.54    | 78.55  | 77.12 | 100.00 | 85.90 | 68.06 | 0.32 | 0.86   | 0.83 | 1.00 | 0.91 | 0.76 |
| USAM_AM    | 42.50    | 84.86  | 82.86 | 98.65  | 90.28 | 77.51 | 0.31 | 0.89   | 0.87 | 0.99 | 0.94 | 0.82 |
| USAM_FS    | 35.00    | 85.58  | 82.41 | 100.00 | 90.88 | 76.99 | 0.37 | 0.90   | 0.87 | 1.00 | 0.95 | 0.82 |
| RSAM_AM    | 16.67    | 66.67  | 65.37 | 93.75  | 72.55 | 60.00 | 0.08 | 0.71   | 0.68 | 0.99 | 0.77 | 0.61 |
| RSAM_FS    | 25.00    | 65.33  | 66.64 | 100.00 | 73.08 | 58.75 | 0.13 | 0.70   | 0.70 | 1.00 | 0.80 | 0.62 |
| DSAM_AM    | 30.00    | 66.67  | 66.15 | 92.11  | 71.43 | 60.30 | 0.22 | 0.71   | 0.69 | 0.98 | 0.77 | 0.63 |
| DSAM_FS    | 15.00    | 65.10  | 64.62 | 90.91  | 70.68 | 59.01 | 0.11 | 0.68   | 0.67 | 0.94 | 0.76 | 0.60 |

signed-rank test to compute statistically significant performance differences between pairings of different sampling techniques with/without

feature selection. The null hypothesis ( $H_0$ ) for the Wilcoxon Sign Rank test states that “there is no significant difference in the performance

**Table 17**

AUC: Statistical and Friedman test results of data sampling with feature selection techniques.

|            | OD_AM | OD_FS | SMOTE_AM | SMOTE_FS | BSMOTE_AM | BSMOTE_FS | SSMOTE_AM | SSMOTE_FS | SMOTEE_AM | SMOTEE_FS | SMOTETO_AM | SMOTETO_FS | USAM_AM | USAM_FS | RSAM_AM | RSAM_FS | DSAM_AM | DSAM_FS | Rank |
|------------|-------|-------|----------|----------|-----------|-----------|-----------|-----------|-----------|-----------|------------|------------|---------|---------|---------|---------|---------|---------|------|
| OD_AM      | 1.00  | 0.00  | 0.00     | 0.00     | 0.00      | 0.00      | 0.00      | 0.00      | 0.00      | 0.00      | 0.00       | 0.00       | 0.00    | 0.00    | 0.98    | 0.21    | 0.99    | 0.00    | 7.56 |
| OD_FS      | 0.00  | 1.00  | 0.00     | 0.00     | 0.00      | 0.00      | 0.00      | 0.00      | 0.00      | 0.00      | 0.00       | 0.00       | 0.00    | 0.00    | 0.00    | 0.00    | 0.00    | 0.00    | 7.62 |
| SMOTE_AM   | 0.00  | 0.00  | 1.00     | 0.27     | 0.19      | 0.25      | 0.05      | 0.30      | 0.00      | 0.00      | 0.03       | 0.00       | 0.00    | 0.00    | 0.00    | 0.00    | 0.00    | 0.00    | 5.03 |
| SMOTE_FS   | 0.00  | 0.00  | 0.27     | 1.00     | 0.01      | 0.97      | 0.00      | 0.95      | 0.00      | 0.00      | 0.00       | 0.00       | 0.00    | 0.00    | 0.00    | 0.00    | 0.00    | 0.00    | 5.13 |
| BSMOTE_AM  | 0.00  | 0.00  | 0.19     | 0.01     | 1.00      | 0.01      | 0.65      | 0.01      | 0.00      | 0.00      | 0.52       | 0.00       | 0.00    | 0.00    | 0.00    | 0.00    | 0.33    | 0.00    | 4.67 |
| BSMOTE_FS  | 0.00  | 0.00  | 0.25     | 0.97     | 0.01      | 1.00      | 0.00      | 0.98      | 0.00      | 0.00      | 0.00       | 0.00       | 0.00    | 0.00    | 0.00    | 0.00    | 0.00    | 0.00    | 5.10 |
| SSMOTE_AM  | 0.00  | 0.00  | 0.05     | 0.00     | 0.65      | 0.00      | 1.00      | 0.00      | 0.00      | 0.00      | 0.92       | 0.00       | 0.00    | 0.00    | 0.00    | 0.00    | 0.00    | 0.00    | 4.48 |
| SSMOTE_FS  | 0.00  | 0.00  | 0.30     | 0.95     | 0.01      | 0.98      | 0.00      | 1.00      | 0.00      | 0.00      | 0.00       | 0.00       | 0.00    | 0.00    | 0.00    | 0.00    | 0.00    | 0.00    | 4.89 |
| SMOTEE_AM  | 0.00  | 0.00  | 0.00     | 0.00     | 0.00      | 0.00      | 0.00      | 0.00      | 1.00      | 0.55      | 0.00       | 0.00       | 0.00    | 0.00    | 0.00    | 0.00    | 0.00    | 0.00    | 1.65 |
| SMOTEE_FS  | 0.00  | 0.00  | 0.00     | 0.00     | 0.00      | 0.00      | 0.00      | 0.00      | 0.55      | 1.00      | 0.00       | 0.00       | 0.00    | 0.00    | 0.00    | 0.00    | 0.00    | 0.00    | 1.70 |
| SMOTETO_AM | 0.00  | 0.00  | 0.03     | 0.00     | 0.52      | 0.00      | 0.92      | 0.00      | 0.00      | 0.00      | 1.00       | 0.00       | 0.00    | 0.00    | 0.00    | 0.00    | 0.00    | 0.00    | 4.10 |
| SMOTETO_FS | 0.00  | 0.00  | 0.00     | 0.00     | 0.00      | 0.00      | 0.00      | 0.00      | 0.00      | 0.00      | 0.00       | 1.00       | 0.00    | 0.00    | 0.00    | 0.00    | 0.00    | 0.00    | 3.81 |
| USAM_AM    | 0.00  | 0.00  | 0.00     | 0.00     | 0.00      | 0.00      | 0.00      | 0.00      | 0.00      | 0.00      | 0.00       | 0.00       | 1.00    | 0.13    | 0.00    | 0.00    | 0.00    | 0.00    | 2.45 |
| USAM_FS    | 0.00  | 0.00  | 0.00     | 0.00     | 0.00      | 0.00      | 0.00      | 0.00      | 0.00      | 0.00      | 0.00       | 0.00       | 0.13    | 1.00    | 0.00    | 0.00    | 0.00    | 0.00    | 2.52 |
| RSAM_AM    | 0.98  | 0.00  | 0.00     | 0.00     | 0.00      | 0.00      | 0.00      | 0.00      | 0.00      | 0.00      | 0.00       | 0.00       | 0.00    | 0.00    | 1.00    | 0.19    | 1.00    | 0.01    | 7.57 |
| RSAM_FS    | 0.21  | 0.00  | 0.00     | 0.00     | 0.00      | 0.00      | 0.00      | 0.00      | 0.00      | 0.00      | 0.00       | 0.00       | 0.00    | 0.00    | 0.19    | 1.00    | 0.23    | 0.00    | 6.80 |
| DSAM_AM    | 0.99  | 0.00  | 0.00     | 0.00     | 0.00      | 0.00      | 0.00      | 0.00      | 0.00      | 0.00      | 0.00       | 0.00       | 0.00    | 0.00    | 1.00    | 0.23    | 1.00    | 0.00    | 7.49 |
| DSAM_FS    | 0.00  | 0.00  | 0.00     | 0.00     | 0.00      | 0.00      | 0.00      | 0.00      | 0.00      | 0.00      | 0.00       | 0.00       | 0.00    | 0.00    | 0.01    | 0.00    | 0.00    | 1.00    | 7.43 |

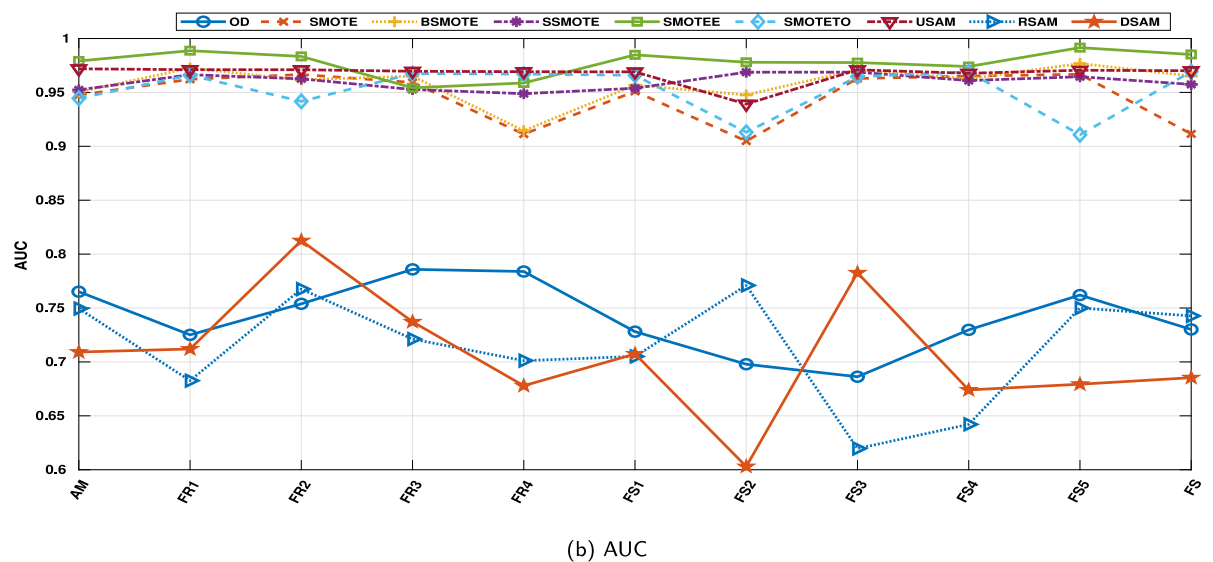
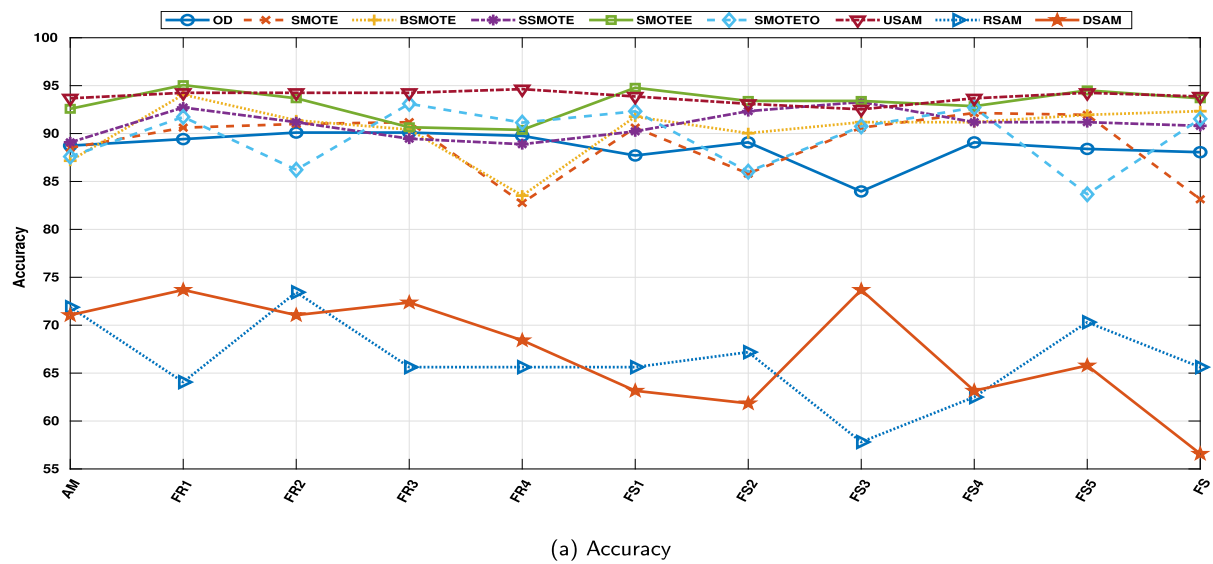


Fig. 11. Accuracy and AUC trends for PR-1: ant-1.5.

of different sampling strategies with/without feature selection". We reject the null hypothesis if  $p \leq 0.05$ . Table 17 demonstrates that most pairs, with  $p$  values less than 0.05, can reject the null hypothesis. As a result, we conclude that the performance of the various sampling approaches with/without different feature selection strategies appears to differ significantly.

The results of the Friedman test for the combination of sampling approaches with/without the application of feature selection are demonstrated in Table 17. To compute it, we separately ranked all sampling techniques with original data, i.e., all metrics (AM), and separately ranked all sampling techniques with feature selection. Table 17 shows that the pair of SMOTEE with FS achieves a mean rank of 1.70, which is reasonably close to the lowest mean rank of 1.65 for the pair of SMOTEE with AM. This proves the optimizing of the performance of the SFP model utilizing sampling techniques with implications of feature selection.

**The results clearly indicates the improved efficiency and predictability of the SFP model post the application of sampling approach with feature selection.**

## 6. Discussion of results for outcomes of combined techniques and individual performance

In the present work, extensive experimentations on 54 open-source projects have been done, and a solution for developing such models to predict the faults with improved performance and predictability power is proposed.

In general, an AUC of 0.7 justifies the trained model as acceptable (Hosmer, Lemeshow, & Sturdivant, 2013), as the experimental results secured from the proposed model show for more than 85% of projects, the trained model delivered a greater than 0.7 AUC value post-application of sampling and feature selection strategies. We will discuss a randomly selected project, say project 1, to show and prove the outcomes. The investigation performed against Project 1 attains, an AUC value greater than 0.7 for 8 applied classifiers with all combinations of 8 sampling and 10 feature selection techniques is 84% (against 80 datasets). The highest possible AUC value for all classifiers with the application of different combinations of sampling and feature selection techniques attained greater than 0.9, this proves the greater predictability of developed SFP models. The RF classifier post-application of FS4 & SMOTEE has outperformed among all other classifiers and combinations with an AUC value of 0.99153.

Table 9 demonstrates that RF performed better than other methods for Project 1. Is this always the case, or does RF performance depend on the right set of features and data sampling techniques? To validate this, we have also displayed the AUC and accuracy values of all combinations of the 11 feature selection and 9 sampling approaches only for RF, the best-performing classifier against Project 1 due to space constraints. The different sampling techniques are represented by different colored lines in Figs. 11(a) and 11(b), where the  $x$ -axis displays the employed feature selection techniques and the  $y$ -axis displays the accuracy and AUC values, respectively. Figs. 11(a) and 11(b), demonstrates that the performance of the SFP model after the implication of various combinations of feature selection and sampling techniques differs significantly. The highest accuracy for Project 1 (ant-1.5) was attained after the implication of FS1 and SMOTEE, as shown in Fig. 11(a), whereas the highest AUC value for Project 1 was attained after the implication of FS5 and SMOTEE, as illustrated in Fig. 11(b). The results demonstrate how the performance of the developed SFP model using the same classifier against the same project is significantly impacted by the various combination of sampling and feature selection approaches. Let us talk about the DSAM sampling method. DSAM and AM together achieve an AUC value of 0.80808 while DSAM and FR1 achieve 0.75252. The following AUC values were obtained for DSAM with FR2, DSAM along FR3, DSAM with FR4, DSAM with FR5, DSAM with FS1, DSAM along

**Table 18**

Best combination of techniques for each project.

| Proj  | Classifier | Sampling | Feat_Sel | AUC  |
|-------|------------|----------|----------|------|
| PR-1  | RF         | SMOTEE   | FS5      | 0.99 |
| PR-2  | RF         | SMOTEE   | FS2      | 0.99 |
| PR-3  | RF         | SMOTEE   | FS5      | 0.99 |
| PR-4  | RF         | SMOTEE   | FR3      | 0.99 |
| PR-5  | RF         | SMOTEE   | FS5      | 0.99 |
| PR-6  | RF         | SMOTEE   | FS2      | 0.96 |
| PR-7  | LOGR       | SMOTEE   | FS2      | 1.00 |
| PR-8  | RF         | SMOTEE   | FS2      | 0.96 |
| PR-9  | RF         | SMOTEE   | AM       | 0.99 |
| PR-10 | RF         | SMOTEE   | FS2      | 0.97 |
| PR-11 | RF         | SMOTEE   | FS2      | 0.99 |
| PR-12 | GNB        | SMOTEE   | FS2      | 1.00 |
| PR-13 | RF         | SMOTEE   | FS2      | 0.99 |
| PR-14 | SVCR       | SMOTEE   | AM       | 1.00 |
| PR-15 | RF         | RSAM     | FS2      | 1.00 |
| PR-16 | DT         | RSAM     | FS4      | 1.00 |
| PR-17 | RF         | SMOTEE   | FS2      | 1.00 |
| PR-18 | GNB        | SMOTEE   | FS2      | 0.99 |
| PR-19 | LOGR       | SMOTEE   | FS2      | 1.00 |
| PR-20 | RF         | SMOTEE   | FR4      | 0.94 |
| PR-21 | LOGR       | SMOTEE   | FS2      | 1.00 |
| PR-22 | DT         | RSAM     | FS1      | 1.00 |
| PR-23 | RF         | SMOTEE   | FS2      | 0.99 |
| PR-24 | RF         | SMOTEE   | FS5      | 0.99 |
| PR-25 | RF         | SMOTEE   | AM       | 1.00 |
| PR-26 | DT         | SMOTEE   | FS5      | 1.00 |
| PR-27 | RF         | SMOTEE   | FR1      | 0.99 |
| PR-28 | RF         | SMOTEE   | FS2      | 0.99 |
| PR-29 | RF         | SMOTEE   | FS5      | 0.90 |
| PR-30 | RF         | SMOTEE   | FS2      | 0.98 |
| PR-31 | RF         | SMOTEE   | FS4      | 0.97 |
| PR-32 | RF         | SMOTEE   | FS2      | 0.99 |
| PR-33 | RF         | SMOTEE   | FS2      | 1.00 |
| PR-34 | RF         | SMOTEE   | FS4      | 0.97 |
| PR-35 | RF         | SMOTE    | FS5      | 0.99 |
| PR-36 | LOGR       | SMOTEE   | FS2      | 1.00 |
| PR-37 | RF         | SMOTEE   | FS4      | 1.00 |
| PR-38 | RF         | SMOTEE   | FS5      | 0.99 |
| PR-39 | DT         | OD       | FS2      | 1.00 |
| PR-40 | RF         | SMOTEE   | FS2      | 1.00 |
| PR-41 | RF         | RSAM     | AM       | 1.00 |
| PR-42 | RF         | SMOTEE   | FS4      | 0.99 |
| PR-43 | RF         | SMOTEE   | FS4      | 1.00 |
| PR-44 | RF         | SMOTEE   | AM       | 1.00 |
| PR-45 | RF         | SMOTEE   | FS5      | 0.98 |
| PR-46 | RF         | SMOTEE   | FS4      | 0.99 |
| PR-47 | RF         | SMOTEE   | FR1      | 0.99 |
| PR-48 | SVCL       | SMOTEE   | FS2      | 1.00 |
| PR-49 | LOGR       | SMOTEE   | FR3      | 1.00 |
| PR-50 | ADAB       | RSAM     | FS2      | 1.00 |
| PR-51 | RF         | SMOTEE   | FS1      | 0.98 |
| PR-52 | GNB        | SMOTEE   | FS2      | 1.00 |
| PR-53 | DT         | SMOTEE   | AM       | 1.00 |
| PR-54 | RF         | SMOTEE   | FS2      | 0.99 |

FS2, DSAM along FS3, DSAM with FS4, and DSAM along FS5: 0.99663, 0.92087, 1, 1, 0.82828, 0.91919, 1, 0.95006, and 0.96408 respectively. Moreover, as mentioned in the 4.2.1 and 4.2.2 sections, the number of extracted features produced using various feature selection techniques differed significantly.

To find the best combination of techniques for software fault prediction, we assessed a total of 792 combinations for each project, which were created using 9 sampling approaches, 11 feature selection methods, and 8 classifiers. As a result, we examined a total of 42768 combinations for the 54 projects that were evaluated. Out of those derived combinations, we have populated the most effective performing combination for each project that has the greater AUC value, in Table 18. We chose the most efficient combination with the fewest amount of input features for projects/datasets when the AUC value was the same for various combinations because it lowers the computational cost of the trained SFP model. In addition, while selecting an efficient

**Table 19**

Friedman test results for all projects.

| PR_ID | Rank_1      | Rank_2      | Rank_3     | Rank_4      | Rank_5      | Rank_6      | Rank_7      | Rank_8      |
|-------|-------------|-------------|------------|-------------|-------------|-------------|-------------|-------------|
| 1     | FS3_SMOTEE  | FS5_SMOTEE  | FR1_SMOTEE | FS6_SMOTEE  | FR2_USAM    | FS1_SMOTEE  | AM_SMOTEE   | FR2_SMOTEE  |
| 2     | FS2_SMOTEE  | AM_SMOTEE   | FR1_SMOTEE | FS3_SMOTEE  | FR2_SMOTEE  | FS1_SMOTEE  | AM_USAM     | FR3_SMOTEE  |
| 3     | FS5_SMOTETO | FS2_SMOTEE  | FS3_SMOTEE | AM_SMOTEE   | FS3_SMOTETO | FS5_SMOTEE  | FR2_SMOTEE  | FR1_SMOTETO |
| 4     | FR4_USAM    | FS3_USAM    | FS6_USAM   | FR2_USAM    | FS6_SMOTEE  | FS5_SMOTEE  | FS1_USAM    | FS4_SMOTEE  |
| 5     | FS3_USAM    | FS4_USAM    | FS5_SMOTEE | FS2_SMOTEE  | FS5_USAM    | AM_USAM     | FS4_SMOTEE  | FR2_SMOTEE  |
| 6     | FS5_SMOTEE  | FS4_SMOTEE  | FS3_SMOTEE | FR1_SMOTEE  | AM_SMOTEE   | FR2_SMOTEE  | FS2_SMOTEE  | FR4_SMOTEE  |
| 7     | FR1_SMOTEE  | FR2_SMOTEE  | FR3_SMOTEE | FR4_SMOTEE  | FS1_SMOTEE  | FS3_SMOTEE  | FS5_SMOTEE  | FS4_SMOTEE  |
| 8     | FS4_USAM    | FS2_SMOTEE  | FS6_USAM   | AM_SMOTEE   | FS5_SMOTEE  | FR1_USAM    | FS5_SMOTETO | FR4_USAM    |
| 9     | FS2_SMOTEE  | AM_SMOTEE   | FR2_SMOTEE | FS4_SMOTEE  | FS1_SMOTEE  | FR3_SMOTEE  | FR1_SMOTEE  | FS6_SMOTEE  |
| 10    | FS4_SMOTEE  | FS3_SMOTEE  | FS5_SMOTEE | FS6_SMOTEE  | AM_SMOTEE   | FS2_SMOTEE  | FR3_SMOTEE  | FR1_SMOTEE  |
| 11    | FS2_SMOTEE  | AM_SMOTEE   | FS4_SMOTEE | FR1_SMOTEE  | FS3_SMOTEE  | FR2_SMOTEE  | FS6_SMOTEE  | FR3_SMOTEE  |
| 12    | AM_SMOTEE   | FR1_SMOTEE  | FS1_SMOTEE | FS4_SMOTEE  | FS5_SMOTEE  | FS6_SMOTEE  | FR3_SMOTEE  | FS2_SMOTEE  |
| 13    | FS2_SMOTEE  | FR3_SMOTEE  | FR2_SMOTEE | FR1_SMOTEE  | FS4_SMOTEE  | FS3_SMOTEE  | AM_SMOTEE   | FS1_SMOTEE  |
| 14    | AM_SMOTEE   | FS6_SMOTEE  | FS5_SMOTEE | FR3_SMOTEE  | FR2_SMOTEE  | FS2_SMOTEE  | FS1_SMOTEE  | FS4_SMOTEE  |
| 15    | FS1_SMOTEE  | FR3_SMOTEE  | FS3_SMOTEE | FR4_SMOTEE  | FS5_SMOTEE  | FS2_BSMOTE  | FS1_DSAM    | FS1_SSMOTE  |
| 16    | FR1_SMOTEE  | FR3_SMOTEE  | FS1_SMOTEE | FS3_SMOTEE  | FR2_SMOTEE  | FS2_SMOTEE  | FS5_SMOTEE  | FR4_SMOTEE  |
| 17    | FS2_SMOTEE  | FS5_SMOTEE  | FS5_RSAM   | FS4_SMOTEE  | FR2_SMOTEE  | FS6_SMOTEE  | FR3_SMOTEE  | FS4_RSAM    |
| 18    | FR2_SMOTEE  | FR1_SMOTEE  | AM_SMOTEE  | FS2_SMOTEE  | FS5_SMOTEE  | FR3_SMOTEE  | FS4_SMOTEE  | FS3_SMOTEE  |
| 19    | FR3_SMOTEE  | FS1_SMOTEE  | FS5_SMOTEE | FS3_SMOTEE  | FR1_SMOTETO | FS2_SMOTEE  | FS3_SMOTETO | AM_SMOTEE   |
| 20    | FS1_SMOTEE  | AM_SMOTEE   | FR3_SMOTEE | FS2_SMOTEE  | FR1_SMOTEE  | FR2_SMOTEE  | FS3_SMOTEE  | FR4_SMOTEE  |
| 21    | FR4_SMOTEE  | FR2_SMOTEE  | FR1_SMOTEE | FS2_SMOTEE  | FS1_SMOTEE  | FS3_SMOTEE  | FS4_SMOTEE  | FS5_SMOTEE  |
| 22    | FS5_USAM    | FS1_USAM    | FR2_USAM   | FR1_USAM    | FR3_USAM    | FS3_USAM    | FS6_USAM    | FR4_USAM    |
| 23    | FS4_USAM    | FR1_USAM    | FS3_USAM   | FR4_USAM    | FS6_USAM    | FR3_USAM    | FS1_USAM    | FS5_USAM    |
| 24    | FS5_SMOTEE  | FS4_USAM    | AM_SMOTEE  | AM_USAM     | FR3_USAM    | FS2_USAM    | FS3_USAM    | FS1_USAM    |
| 25    | AM_SMOTEE   | FR1_SMOTEE  | FS3_SMOTEE | FR2_SMOTEE  | FS4_SMOTEE  | FS1_SMOTEE  | FR3_SMOTEE  | AM_SMOTETO  |
| 26    | FR1_SMOTEE  | FS3_SMOTEE  | FS5_SMOTEE | FR3_SMOTEE  | FR4_SMOTETO | FS3_SMOTETO | FS6_SMOTETO | FS1_SMOTETO |
| 27    | FR3_SMOTEE  | FR4_USAM    | FS4_USAM   | FR2_USAM    | FR2_SMOTEE  | FR3_USAM    | FS4_SMOTEE  | FS6_USAM    |
| 28    | FS2_SMOTEE  | FS4_USAM    | FS5_USAM   | FR1_USAM    | AM_SMOTEE   | FS3_USAM    | FR4_USAM    | FS1_USAM    |
| 29    | FS4_USAM    | FS6_USAM    | FS2_SMOTEE | FS3_USAM    | AM_USAM     | FR2_SMOTEE  | FS5_USAM    | FR2_USAM    |
| 30    | FS4_SMOTEE  | FS2_SMOTEE  | FS3_SMOTEE | AM_SMOTEE   | FR1_SMOTEE  | FS5_SMOTEE  | FS6_SMOTEE  | FR4_SMOTEE  |
| 31    | FS4_USAM    | FS3_USAM    | FS6_USAM   | FR2_USAM    | FR4_USAM    | FS1_USAM    | FS4_SMOTEE  | FR3_USAM    |
| 32    | FS5_SMOTEE  | FR2_SMOTEE  | AM_SMOTEE  | FR3_SMOTEE  | FS4_SMOTEE  | FS2_USAM    | FS4_USAM    | FS3_USAM    |
| 33    | FS2_SMOTEE  | FR2_SMOTEE  | FS4_SMOTEE | FS1_SMOTEE  | FR3_SMOTEE  | FR1_SMOTEE  | FS3_SMOTEE  | FS6_SMOTEE  |
| 34    | FS4_USAM    | FR2_USAM    | FS6_USAM   | FS1_USAM    | FS3_SMOTEE  | FR4_USAM    | FS4_SMOTEE  | FR3_USAM    |
| 35    | FR1_USAM    | FS4_USAM    | FS5_USAM   | FR3_USAM    | FR4_USAM    | FS3_USAM    | AM_BSMOTE   | FS1_USAM    |
| 36    | FR3_SMOTEE  | FS3_SMOTEE  | FS4_SMOTEE | FS5_SMOTEE  | FS6_SMOTEE  | FR2_SMOTEE  | FS2_SMOTEE  | FS1_SMOTEE  |
| 37    | FS4_USAM    | FS1_USAM    | FR3_USAM   | FS6_USAM    | FS5_USAM    | FR1_USAM    | FR2_USAM    | FR4_USAM    |
| 38    | FS4_USAM    | FR2_USAM    | FR1_USAM   | FS5_USAM    | FR3_USAM    | FS1_USAM    | FS5_SMOTEE  | FS6_USAM    |
| 39    | FS2_OD      | FR3_USAM    | FS1_USAM   | FR4_USAM    | FR2_USAM    | FS4_USAM    | FS5_USAM    | FS2_SMOTEE  |
| 40    | FR3_SMOTEE  | FS1_SMOTEE  | FS5_SMOTEE | FR1_SMOTEE  | FS5_BSMOTE  | FS1_USAM    | FS4_SMOTE   | FR3_BSMOTE  |
| 41    | FS3_USAM    | FR1_USAM    | FR2_USAM   | FS5_USAM    | FS6_USAM    | FR4_USAM    | FR3_USAM    | FS1_USAM    |
| 42    | FS4_USAM    | FR1_USAM    | FR4_USAM   | FS1_USAM    | FR2_USAM    | FS2_SMOTEE  | FS4_SMOTEE  | FS3_USAM    |
| 43    | FS4_USAM    | FR1_USAM    | FR2_USAM   | FS1_USAM    | FS5_SMOTEE  | FS2_SMOTEE  | FS4_SMOTEE  | FS5_USAM    |
| 44    | FS2_SMOTEE  | AM_SMOTEE   | FS6_SMOTEE | FS3_SMOTEE  | FS5_SMOTEE  | FR2_SMOTEE  | FS4_SMOTEE  | FR3_SMOTEE  |
| 45    | FS5_SMOTEE  | FS2_SMOTEE  | AM_SMOTEE  | FS4_SMOTEE  | FS4_USAM    | FR2_SMOTEE  | FS3_SMOTEE  | FS3_USAM    |
| 46    | FS4_USAM    | FR1_USAM    | FS1_USAM   | FS6_USAM    | FS5_USAM    | FR2_USAM    | FS4_SMOTEE  | FS6_SMOTEE  |
| 47    | AM_SMOTEE   | FS6_SMOTEE  | FS2_SMOTEE | FS5_SMOTEE  | FR2_SMOTEE  | FR1_SMOTEE  | FS3_SMOTEE  | FS4_SMOTEE  |
| 48    | FS1_SMOTETO | FR4_SMOTETO | FS5_SMOTEE | FR1_SMOTETO | FS1_SMOTE   | FR4_USAM    | FR3_BSMOTE  | FR2_SMOTEE  |
| 49    | FR3_SMOTEE  | AM_SMOTEE   | FS4_SMOTEE | FS6_SMOTEE  | FS2_SMOTEE  | FS5_SMOTEE  | FS1_SMOTEE  | FS3_SMOTETO |
| 50    | FS3_SMOTEE  | FS6_SMOTEE  | FS1_SMOTEE | AM_SMOTEE   | FR4_SMOTEE  | FS5_SMOTEE  | FR3_RSAM    | FS4_RSAM    |
| 51    | FS2_SMOTEE  | AM_SMOTEE   | FR1_SMOTEE | FS1_SMOTEE  | FS4_SMOTEE  | FS5_SMOTEE  | FR4_SMOTEE  | FR2_SMOTEE  |
| 52    | FS2_SMOTEE  | FS5_SMOTEE  | AM_SMOTEE  | FR2_SMOTEE  | FS4_SMOTEE  | FR3_SMOTEE  | FR4_SMOTEE  | FR1_SMOTEE  |
| 53    | FS2_SMOTEE  | AM_SMOTEE   | FS6_SMOTEE | FS5_SMOTEE  | FR3_SMOTEE  | FS3_SMOTEE  | FR4_SMOTEE  | FS1_SMOTEE  |
| 54    | FS2_SMOTEE  | AM_SMOTEE   | AM_USAM    | FS5_SMOTEE  | FR2_SMOTEE  | FS5_USAM    | FS6_SMOTEE  | FS2_SSMOTE  |

combination, in case of the same AUC value for various combinations the weightage given to those models comprises unsampled or original data over-sampling data, as the implementation of the sampling approach will also increase complexity. From Table 18, we observed that out of the 54 projects, no SFP model that combines all feature metrics and original datasets performs better than the others. There is only one project i.e., project 39, where we got the best model for an original dataset with the FS2 feature selection approach. Additionally, in the majority of the projects, we found that using a combination of feature selection and data sampling techniques yielded the best results. Table 18, depicts the SMOTEE and FS2 combination outperformed with the highest AUC value for 38.89% of projects i.e., against 21 projects among 54 projects. Additionally, experimental findings indicate that for 68.51% of projects, the RF classifier achieved the greatest AUC value. 24.07% of projects with the combination of SMOTEE, FS2, and RF, attained the highest AUC values. Furthermore, the results of

the Friedman test for all 54 projects are displayed in Table 19. We computed the top 8 ranks for each project using 99 combinations of feature selection and sampling techniques (11 (10 feature selection methods + 1 all metric) \* 9 (8 sampling methods + 1 unsampled dataset)). Table 19 shows that only four projects 12, 14, 25, and 47 out of the 54 total projects achieved the best lowest mean rank using a combination of AM and SMOTEE. For the remaining projects, the performance of the SFP model has improved due to the combination of feature selection and sampling approaches. The Friedman test results demonstrated how the SFP model's performance was impacted by a particular combination of feature selection and sampling approaches.

The obtained outcomes of the SFP model trained post application of sampling and feature selection techniques, indicate the improved AUC by a range of 11% to 40%. We will talk about a randomly chosen project, say project 39, to demonstrate and prove. The RF classifier against project 39, achieved an AUC value of 0.74866 without the

implication of sampling and feature selection techniques. Whereas after balancing Project 39's dataset using the SMOTEE technique and considering all metrics the trained model using RF provided the AUC value as 0.97142, this expresses the 23% improved predictability. Furthermore, employment of the FS-1 feature selection technique against balanced data of project 39 using SMOTEE shows elevated improvement of AUC value as 0.98752 of trained SFP model using RF classifier. During this, the study shows after applying the FS-1 feature selection method, 75% of irrelevant features have been removed; still, the trained model gives better results; this manifests the feature selection addresses the feature redundancy problem.

The result shows after the implication of SMOTEE and FS-1, the trained SFP model's accuracy value improved significantly. To validate our claim, we will talk about a random project, say project 48. For the 48th project, the accuracy of GNB, LOGR, DT, RF, SVCR, SVCL, BAG, and ADAB classifiers have improved by 50%, 20.55%, 60%, 50%, 56.30%, 20.56%, 26.48%, & 35% respectively. Whereas the accuracy of GNB, LOGR, DT, RF, SVCR, SVCL, and ADAB classifiers against the same project has been intensified by 16.67, 35.84, 55.84, 45.84, 26.67, 23.33, 22.50 respectively, however, we detect a 9.16 percentage decrease in accuracy for BAG after the implication of SMOTE and FS-1. The intensive experiments performed on 54 projects shows the improved accuracy & AUC of all eight classifiers with the employment of the SMOTEE sampling technique. Applying the SMOTEE sampling technique and FS-1 feature selection technique for project 12, the observed value of accuracy & AUC of all classifiers have been the same as with considering all metrics. The sampling technique improves defect predictability, and feature selection techniques boost the performance efficiency of the SFP model. The experimental result shows the Random Forest ensemble gives a better performance.

## 7. Threats to validity

**Internal Validity:** The faulty software metrics datasets utilized for evaluating the suggested models are acquired from the tera-promise data repository. Although we cannot ensure that the datasets used in this study are 100% accurate, we believe it has been obtained consistently. In other contexts, concerning internal validity, it is worth noting that the balanced datasets produced by data sampling techniques are not an exact reproduction of the original dataset. We may not have generalized this balanced dataset for testing while using it to train SFP models; this could lead to validation bias and compromise internal validity. The study, on the other hand, analyses the predictions of the SFP model using the robust performance measures AUC and accuracy, which reduces validation bias. Furthermore, we performed nonparametric tests such as the Friedman test and the Wilcoxon Signed Rank Sum test to determine the statistical significance of the results.

**Construct Validity:** Construct validity assures that the method of assessing the input and output parameters of the SFP model is correct. As noted in the related work section, many scholars have already built SFP prediction using various sets of source code metrics, which we have chosen as input datasets. Also, the employed classifiers in this research are the widely studied classifiers. Therefore, we may be confident that construct validity risks have been eliminated from our work. As a result, any challenges to constructing validity may not be present in our work.

**External Validity:** The term "external validity" relates to whether the study can be applied widely. The datasets utilized in this research are open-source object-oriented datasets written in Java language. Hence, the study may have different outcomes for other programming language datasets. As a result, there is a risk of external validity in this study. The arguments setting of implemented models, on the other hand, helps to lessen generalizability threats.

## 8. Conclusion & future work

The SFP model helps in building quality software with minimal testing efforts by detecting faulty software components at the initial stage of Software Development. This research represents a significant step forward in the development of effective SFP models by dealing with the class imbalance and feature redundancy problem efficiently. The developed SFP models using 8 different classifiers are analyzed and their performance has been measured against 54 open-source datasets. The study employs 8 data resampling strategies and 10 feature selection techniques to resolve the problem of data correlation, redundancy, feature irrelevance, and imbalanced data. The SFP models were validated using accuracy & AUC. The research outcomes are statistically analyzed with the Wilcoxon Sign Rank test and Friedman test. The focused insights of this research are:

- The empirical finding shows that SMOTEE with FS2 combination outperformed with the highest AUC value, for 38.89% of projects
- For 90% of projects, the adoption of sampling methods such as SMOTEE, SMOTETO, and USAM improves the predictability of developed SFP models using 8 classifiers
- Employing the feature selection strategies over balanced datasets, reduces computational effort and improves the overall performance of SFP model's against more than 80% of projects
- In comparison to other employed sampling techniques to handle the data imbalance, the SMOTEE technique outperformed by gaining the highest median accuracy & AUC of 87.15% & 0.93 respectively
- Study observed, that the FR5 feature ranking technique performance excels in comparison to other employed feature selection techniques by attaining 76.61% accuracy & 0.82 AUC
- The Friedman test, the USAM sampling technique obtained the second-highest mean rank with 2.87, next in line to SMOTEE among other sampling strategies
- The RF classifier stands first among all other classifiers by acquiring a 1.93 mean rank value
- Post implication of feature selection techniques study indicates, a reduction of 75% irrelevant features from the original dataset while pertaining improved performance of SFP models trained against all metrics

Even though the datasets are imbalanced, the developed SFP model can be used by software practitioners to efficiently predict faulty software components. This research work supports the application of the SMOTEE sampling technique and Correlation-based feature selection, and Chi-squared test feature selection methods to develop an efficient SFP model. The forecast for future work to the best of our knowledge is:

- In this study, the engaged ensemble classifiers are the most widely studied. In the future, deep learning algorithms will be followed to train SFP models.
- More imbalanced datasets created in any popular languages other than Java will be included, and the influence of sampling approaches and feature selection strategies will be investigated.
- During this study, CKJM (Chidamber and Kemerer Java Metrics) tool is used to calculate the metrics; hence the developed model may or may not work for the projects in which the metrics are calculated using a different tool other than CKJM. In future studies, AST (Abstract Syntax Tree) technique would be used for metrics calculation.

## CRedit authorship contribution statement

**Sonika Chandrakant Rathi:** Conceptualization, Methodology, Validation, Investigation, Writing – original draft. **Sanjay Misra:** Conceptualization, Methodology, Validation, Investigation, Resources, Writing

– original draft, Supervision. **Ricardo Colomo-Palacios:** Methodology, Validation, Writing – review & editing. **R. Adarsh:** Methodology, Investigation, Writing – review & editing. **Lalita Bhanu Murthy Neti:** Methodology, Validation, Resources, Writing – review & editing, Supervision. **Lov Kumar:** Conceptualization, Methodology, Validation, Investigation, Resources, Writing – review & editing, Supervision.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

Data will be made available on request.

## References

- Arshad, A., Riaz, S., Jiao, L., & Murthy, A. (2018). Semi-supervised deep fuzzy c-mean clustering for software fault prediction. *IEEE Access*, 6, 25675–25685.
- Aziz, S. R., Khan, T., & Nadeem, A. (2019). Experimental validation of inheritance metrics' impact on software fault prediction. *IEEE Access*, 7, 85262–85275.
- Balogun, A. O., Lafenwa-Balogun, F. B., Mojeed, H. A., Usman-Hamza, F. E., Bajeh, A. O., Adeyemo, V. E., et al. (2020). Data sampling-based feature selection framework for software defect prediction. In *The international conference on emerging applications and technologies for industry 4.0* (pp. 39–52). Springer.
- Basili, V. R., Briand, L. C., & Melo, W. L. (1996). A validation of object-oriented design metrics as quality indicators. *IEEE Transactions on Software Engineering*, 22(10), 751–761.
- Bhandari, K., Kumar, K., & Sangal, A. L. (2021). A study on modeling techniques in software fault prediction. In *2021 2nd international conference on secure cyber computing and communications (ICSCCC)* (pp. 6–11). IEEE.
- Chen, J., Liu, S., Liu, W., Chen, X., Gu, Q., & Chen, D. (2014). A two-stage data preprocessing approach for software fault prediction. In *2014 eighth international conference on software security and reliability (SERE)* (pp. 20–29). IEEE.
- Chidamber, S. R., & Kemerer, C. F. (1994). A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6), 476–493.
- Crasso, M., Mateos, C., Zunino, A., Misra, S., & Polvorín, P. (2016). Assessing cognitive complexity in java-based object-oriented systems: Metrics and tool support. *Computing and Informatics*, 35(3), 497–527.
- Cynthia, S. T., & Ripon, S. H. (2019). Predicting and classifying software faults: A data mining approach. In *Proceedings of the 2019 7th international conference on computer and communications management* (pp. 143–147).
- Dam, H. K., Pham, T., Ng, S. W., Tran, T., Grundy, J., Ghose, A., et al. (2018). A deep tree-based model for software defect prediction. arXiv preprint arXiv:1802.00921.
- Elahi, E., Kanwal, S., & Asif, A. N. (2020). A new ensemble approach for software fault prediction. In *2020 17th international bhurban conference on applied sciences and technology (IBCAST)* (pp. 407–412). IEEE.
- Gao, K., Khoshgoftaar, T. M., Wang, H., & Seliya, N. (2011). Choosing software metrics for defect prediction: an investigation on feature selection techniques. *Software - Practice and Experience*, 41(5), 579–606.
- Gong, L., Jiang, S., & Jiang, L. (2019). Tackling class imbalance problem in software defect prediction through cluster-based over-sampling with filtering. *IEEE Access*, 7, 145725–145737.
- Hall, M. A., & Holmes, G. (2003). Benchmarking attribute selection techniques for discrete class data mining. *IEEE Transactions on Knowledge and Data Engineering*, 15(6), 1437–1447.
- Hall, M. A., et al. (1999). Correlation-based feature selection for machine learning.
- Henderson-Sellers, B. (1996). The mathematical validity of software metrics. *ACM SIGSOFT Software Engineering Notes*, 21(5), 89–94.
- Hosmer, D. W., Jr., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied logistic regression*, Vol. 398. John Wiley & Sons.
- Hou, S., & Li, Y. (2009). Short-term fault prediction based on support vector machines with parameter optimization by evolution strategy. *Expert Systems with Applications*, 36(10), 12383–12391.
- Ji, H., Huang, S., Wu, Y., Hui, Z., & Zheng, C. (2019). A new weighted naive Bayes method based on information diffusion for software defect prediction. *Software Quality Journal*, 27(3), 923–968.
- Joon, A., Tyagi, R. K., & Kumar, K. (2020). Noise filtering and imbalance class distribution removal for optimizing software fault prediction using best software metrics suite. In *2020 5th international conference on communication and electronics systems (ICCES)* (pp. 1381–1389). IEEE.
- Jureczko, M. (2011). Significance of different software metrics in defect prediction. *Software Engineering: An International Journal*, 1(1), 86–95.
- Karim, S., Warnars, H. L. H. S., Gaol, F. L., Abdurachman, E., Soewito, B., et al. (2017). Software metrics for fault prediction using machine learning approaches: A literature review with PROMISE repository dataset. In *2017 IEEE international conference on cybernetics and computational intelligence (CyberneticsCom)* (pp. 19–23). IEEE.
- Khuat, T. T., & Le, M. H. (2019). Ensemble learning for software fault prediction problem with imbalanced data. *International Journal of Electrical and Computer Engineering*, 9(4), 3241.
- Kondo, M., Bezemer, C.-P., Kamei, Y., Hassan, A. E., & Mizuno, O. (2019). The impact of feature reduction techniques on defect prediction models. *Empirical Software Engineering*, 24(4), 1925–1963.
- Kulamala, V. K., Kumar, L., & Mohapatra, D. P. (2021). Software fault prediction using LSSVM with different Kernel functions. *Arabian Journal for Science and Engineering*, 46(9), 8655–8664.
- Kumar, L., Sripada, S. K., Sureka, A., & Rath, S. K. (2018). Effective fault prediction model developed using least square support vector machine (LSSVM). *Journal of Systems and Software*, 137, 686–712.
- Laradji, I. H., Alshayeb, M., & Ghouti, L. (2015). Software defect prediction using ensemble learning on selected features. *Information and Software Technology*, 58, 388–402.
- Lessmann, S., Baesens, B., Mues, C., & Pietsch, S. (2008). Benchmarking classification models for software defect prediction: A proposed framework and novel findings. *IEEE Transactions on Software Engineering*, 34(4), 485–496.
- Liu, W., Liu, S., Gu, Q., Chen, J., Chen, X., & Chen, D. (2015). Empirical studies of a two-stage data preprocessing approach for software fault prediction. *IEEE Transactions on Reliability*, 65(1), 38–53.
- Liu, H., & Yu, L. (2005). Toward integrating feature selection algorithms for classification and clustering. *IEEE Transactions on Knowledge and Data Engineering*, 17(4), 491–502.
- Malhotra, R., & Jain, A. (2012). Fault prediction using statistical and machine learning methods for improving software quality. *Journal of Information Processing Systems*, 8(2), 241–262.
- Malhotra, R., & Kamal, S. (2019). An empirical study to investigate oversampling methods for improving software defect prediction using imbalanced data. *Neurocomputing*, 343, 120–140.
- Mangla, M., Sharma, N., & Mohanty, S. N. (2021). A sequential ensemble model for software fault prediction. *Innovations in Systems and Software Engineering*, 1–8.
- McCabe, T. J. (1976). A complexity measure. *IEEE Transactions on Software Engineering*, 4(4), 308–320.
- Menzies, T., Milton, Z., Turhan, B., Cukic, B., Jiang, Y., & Bener, A. (2010). Defect prediction from static code features: current results, limitations, new approaches. *Automated Software Engineering*, 17(4), 375–407.
- Misra, S., Adewumi, A., Fernandez-Sanz, L., & Damasevicius, R. (2018). A suite of object oriented cognitive complexity metrics. *IEEE Access*, 6, 8782–8796.
- Mohapatra, Y., & Ray, M. (2018). Software fault prediction based on GSOGA optimization with kernel based SVM classification. *International Journal of Intelligent Systems*, 5(11).
- Nevendra, M., & Singh, P. (2021). Software defect prediction by strong machine learning classifier. In *Intelligent computing and communication systems* (pp. 321–329). Springer.
- Pak, C., Wang, T. T., & Su, X. H. (2018). An empirical study on software defect prediction using over-sampling by SMOTE. *International Journal of Software Engineering and Knowledge Engineering*, 28(06), 811–830.
- Pandey, S. K., Mishra, R. B., & Tripathi, A. K. (2021). Machine learning based methods for software fault prediction: A survey. *Expert Systems with Applications*, 172, Article 114595.
- Pandey, S. K., & Tripathi, A. K. (2021). An empirical study toward dealing with noise and class imbalance issues in software defect prediction. *Soft Computing*, 25(21), 13465–13492.
- Putri, S. A., et al. (2017). Combining integrated sampling technique with feature selection for software defect prediction. In *2017 5th international conference on cyber and it service management (CITSM)* (pp. 1–6). IEEE.
- Ranjan, P., Kumar, S., & Kumar, U. (2017). Software fault prediction using computational intelligence techniques: A survey. *Indian Journal of Science and Technology*, 10(18), 1–9.
- Rathore, S. S., & Gupta, A. (2014). A comparative study of feature-ranking and feature-subset selection techniques for improved fault prediction. In *Proceedings of the 7th India software engineering conference* (pp. 1–10).
- Rhmann, W., Pandey, B., Ansari, G., & Pandey, D. K. (2020). Software fault prediction based on change metrics using hybrid algorithms: An empirical study. *Journal of King Saud University-Computer and Information Sciences*, 32(4), 419–424.
- Rodriguez, D., Ruiz, R., Cuadrado-Gallego, J., & Aguilar-Ruiz, J. (2007). Detecting fault modules applying feature selection to classifiers. In *2007 IEEE international conference on information reuse and integration* (pp. 667–672). IEEE.
- Shao, Y., Liu, B., Wang, S., & Li, G. (2020). Software defect prediction based on correlation weighted class association rule mining. *Knowledge-Based Systems*, 196, Article 105742.
- Shatnawi, R. (2017). The application of ROC analysis in threshold identification, data imbalance and metrics selection for software fault prediction. *Innovations in Systems and Software Engineering*, 13(2), 201–217.

- Singh, Y., & Malhotra, R. (2012). *Object-oriented software engineering*. PHI Learning Pvt. Ltd..
- Son, L. H., Pritam, N., Khari, M., Kumar, R., Phuong, P. T. M., Thong, P. H., et al. (2019). Empirical study of software defect prediction: a systematic mapping. *Symmetry*, 11(2), 212.
- Tan, P.-N., Steinbach, M., & Kumar, V. (2005). *Introduction to data mining* (1st ed.). USA: Addison-Wesley Longman Publishing Co., Inc..
- Tantithamthavorn, C., Hassan, A. E., & Matsumoto, K. (2018). The impact of class rebalancing techniques on the performance and interpretation of defect prediction models. *IEEE Transactions on Software Engineering*, 46(11), 1200–1219.
- Tantithamthavorn, C., Hassan, A. E., & Matsumoto, K. (2020). The impact of class rebalancing techniques on the performance and interpretation of defect prediction models. *IEEE Transactions on Software Engineering*, 46(11), 1200–1219.
- Thaher, T., Mafarja, M., Abdalhaq, B., & Chantar, H. (2019). Wrapper-based feature selection for imbalanced data using binary queuing search algorithm. In *2019 2nd international conference on new trends in computing sciences (ICTCS)* (pp. 1–6). IEEE.
- Tubishat, M., Idris, N., Shuib, L., Abushariah, M. A., & Mirjalili, S. (2020). Improved salp swarm algorithm based on opposition based learning and novel local search algorithm for feature selection. *Expert Systems with Applications*, 145, Article 113122.
- Turabieh, H., Mafarja, M., & Li, X. (2019). Iterated feature selection algorithms with layered recurrent neural network for software fault prediction. *Expert Systems with Applications*, 122, 27–42.
- Wang, H., Khoshgoftaar, T. M., & Napolitano, A. (2010). A comparative study of ensemble feature selection techniques for software defect prediction. In *2010 ninth international conference on machine learning and applications* (pp. 135–140). IEEE.
- Weiss, G. M., McCarthy, K., & Zabar, B. (2007). Cost-sensitive learning vs. sampling: Which is best for handling unbalanced classes with unequal error costs? *Dmin*, 7(35–41), 24.
- Woolson, R. F. (2007). Wilcoxon signed-rank test. *Wiley Encyclopedia of Clinical Trials*, 1–3.
- Yohannese, C. W., & Li, T. (2017). A combined-learning based framework for improved software fault prediction. *International Journal of Computational Intelligence Systems*, 10(1), 647.
- Zhou, T., Sun, X., Xia, X., Li, B., & Chen, X. (2019). Improving defect prediction with deep forest. *Information and Software Technology*, 114, 204–216.
- Zimmerman, D. W., & Zumbo, B. D. (1993). Relative power of the Wilcoxon test, the Friedman test, and repeated-measures ANOVA on ranks. *The Journal of Experimental Education*, 62(1), 75–86.
- Zong, P., Wang, Y., & Xie, F. (2018). Embedded software fault prediction based on back propagation neural network. In *2018 IEEE international conference on software quality, reliability and security companion (QRS-C)* (pp. 553–558). IEEE.